



BACS2063 Data Structures and Algorithms

ASSIGNMENT 202501

Declaration

- I confirm that I have read and complied with all the terms and conditions of Tunku Abdul Rahman University of Management and Technology's plagiarism policy.
- I declare that this assignment is free from all forms of plagiarism and for all intents and purposes is my own properly derived work.

Student Name	Student ID	Prog / Tut.Grp	Signature
Edison Chan Yoong Qi	24WMR08859	RDS2S3 G4	<i>Chan</i>
Khiew Jit Chen	24WMR08869	RDS2S3 G4	<i>Khiew</i>
Ethel Ng Yi Yan	24WMR08861	RDS2S3 G4	<i>Ethel</i>
Hea Hui Min	24WMR08865	RDS2S3 G4	<i>Min</i>

Note: The submission date and time will be according to the timestamp recorded in Google Classroom for Assignment Submission.

Edison Chan Yoong Qi	Khiew Jit Chen	Ethel Ng Yi Yan	Hea Hui Min
			

Table of Contents

Part A: TEAM REPORT	3
1. Abstract Data Type (ADT)	3
1.1 ADT Specification	3
1.1.1 Sorted Doubly Linked List ADT Specification	3
1.2 ADT Implementation	10
1.2.1 Sorted List Interface	10
1.2.2 Sorted Doubly Linked List	12
Implementation Class: SortedDoublyLinkedList.java	12
Part B. INDIVIDUAL REPORT	26
2. Use of ADTs	26
Edison Chan Yoong Qi	26
Business Rules	26
Source codes for Control classes	26
Screenshots	47
Khiew Jit Chen	52
Business Rules for Application Management	52
Source codes for Control classes	52
Screenshots	67
Ethel Ng Yi Yan	73
Business Rules for Job Management Module	73
1. Source codes for Control classes	73
2. Screenshots	91
Hea Hui Min	112
Business Rules for Interview Management	112
1. Source codes for Control classes	112
2. Screenshots	125

Part A: TEAM REPORT

1. Abstract Data Type (ADT)

1.1 ADT Specification

SortedDoublyLinkedList

A SortedLinkedList is a linear collection of unique elements of a generic type T which does not allow duplicate elements to be arranged in a sorted order. Position start from 1

1.1.1 Sorted Doubly Linked List ADT Specification

Function	integer getSize()
Description	Returns the size of the Doubly Linked List
Pre-condition	-
Post-condition	List remains unchanged
Return	Returns the number of records in the Doubly Linked List

Function	boolean isEmpty()
Description	Checks whether if the number of entries in the list is 0
Pre-condition	-
Post-condition	List remains unchanged
Return	True if the list is empty. False if list is not empty

Function	insertAtHead(T data)
Description	Adds the data into the list at the head
Pre-condition	Checks if the current list is empty
Post-condition	Data is added into the list at head
Return	-

Function	insertAtTail(T data)
Description	Adds the data into the list at the tail
Pre-condition	Checks if the current list is empty
Post-condition	Data is added into the list at the tail
Return	-

Function	insertAtIndex(int index, T data)
Description	Insert data into list, with index given
Pre-condition	Checks whether int index fits within the range of the list
Post-condition	Data is added into the list at the specified index
Return	-

Function	deleteFromHead()
Description	Deletes the data from the list at its head
Pre-condition	Checks whether the list is empty
Post-condition	Data at head is removed, list sizes is decremented
Return	-

Function	deleteFromTail()
Description	Deletes the data from the list at its tail
Pre-condition	Checks whether the list is empty
Post-condition	Data at tail is removed, list sizes is decremented
Return	-

Function	displayForward()
Description	Display all the data in the list, from head to tail
Pre-condition	Checks whether the list is empty

Post-condition	Data is displayed in the list, from head to tail, if no records exist, will return a message saying list is empty
Return	-

Function	displayBackward()
Description	Display all the data in the list, from tail to head
Pre-condition	Checks whether the list is empty
Post-condition	Data is displayed in the list, from tail to head, if no records exist, will return a message saying list is empty
Return	-

Function	deleteNode(T data)
Description	Deletes data that matches with the one in the list
Pre-condition	Check whether the list is empty.
Post-condition	Data is deleted from the list
Return	-

Function	Iterator<T> iterator()
Description	Iterates through the list and print data
Pre-condition	Checks whether current is not null
Post-condition	List remains unchanged during iteration
Return	Data in the list

Function	String toString()
Description	Prints data from a list within a list
Pre-condition	Checks whether if the list exists or is empty

Post-condition	List is unchanged
Return	Data as a formatted String

Function	insertSorted(T data)
Description	Inserts element while maintaining sort order
Pre-condition	Checks if list is empty for head, tail reference
Post-condition	Data is added into the list, sorted
Return	-

Function	mergeSortData()
Description	Uses recursion to sort the list
Pre-condition	Checks if list is empty
Post-condition	List is sorted according to the comparator in the entity class
Return	Returns a sorted list, replacing the existing list

Function	updateData(T data, int index)
Description	Update data based on the specific index
Pre-condition	Checks the given index is within the valid range
Post-condition	The data at the specific index in the list is updated to new value
Return	-

Function	swap(int index1, int index2)
Description	Exchange the data stored at two specific position in the list
Pre-condition	Checks the both index within the valid size of the list and check both

	index must not identical
Post-condition	The data from both specific indexes have been swapped.
Return	-

Function	T atIndex(int index)
Description	Get data at a specific index
Pre-condition	Checks whether if the index given is within bounds of the list
Post-condition	List is unchanged, data at index n is returned
Return	Data in the list

Function	boolean contains(T data)
Description	Check whether specific data exists in the list
Pre-condition	Checks whether if the list is empty
Post-condition	List is unchanged
Return	True if the list contains specific data. False if the list does not contains specific data

Function	SortedListInterface<T> intersect(SortedListInterface<T> otherList)
Description	Find and return the similar element in both list
Pre-condition	both this list and otherList must be of the same generic type T T must implement Comparable<T>
Post-condition	The current list is replaced
Return	a sorted list containing elements present in both lists

Function	SortedListInterface<T> cloneList()
Description	Creates and return a new list containing same data stored in the current list
Pre-condition	Checks empty list
Post-condition	A new list is cloned
Return	Object that is the copy of the current list

Function	SortedListInterface<T> extractByAttributes(SortedListInterface<T> list, String attribute)
Description	Returns a list of data where the data fields contains the selected attribute
Pre-condition	Checks if the list is an empty input and empty string
Post-condition	Replaces the current list with a new list containing only the selected attribute
Return	Returns the current list with a new list containing only the selected attribute

Function	SortedListInterface<T> getFirstN(int n)
Description	Returns the first N data within a list
Pre-condition	Checks if int N is out of bounds and whether if list is empty
Post-condition	Replaces the current list with a new list containing only the first N data
Return	Returns the current list with a new list containing only the first N data

Function	SortedListInterface<String> extractUniqueValues(String fieldName)
----------	---

Description	Extract every unique values from a given fieldName from a list
Pre-condition	Check if list is empty
Post-condition	Replaces the current list with a String list containing every unique values in a given field in the list
Return	Returns a String list containing every unique values in a given field in the list

Function	boolean containsByName(T data, String field_Name)
Description	Iterates through the list to find duplicate data in any of the selected data field
Pre-condition	Checks if list is empty
Post-condition	List is unchanged
Return	Returns true if there exist a duplicate between T data and in the list, otherwise return false

1.2 ADT Implementation

1.2.1 Sorted List Interface

Interface Class: SortedListInterface.java

```
package Adt;

import java.util.Iterator;

// Author: Edison Chan Yoong Qi, Khiew Jit Chen, Ethel Ng Yi Yan, Hea Hui Min
public interface SortedListInterface<T> extends Iterable<T> {

    // Return size of list
    public int getSize();

    // Checks if empty
    public boolean isEmpty();

    // Inserts at head
    public void insertAtHead(T data);

    // Insert at tail
    public void insertAtTail(T data);

    // Insert at index
    public void insertAtIndex(int index, T data);

    // Delete Head
    public void deleteFromHead();

    // Delete Tail
    public void deleteFromTail();

    // Display next node
    public void displayForward();

    // Display prev node
    public void displayBackward();

    // Delete the selected node
    public void deleteNode(T data);

    // Allow iteration over the list
    @Override
    public Iterator<T> iterator();

    // Prints out list data within the list
    @Override
    public String toString();
```

```

// Insert while maintaining sort order
public void insertSorted(T data);

// Initiate merge sort, uses recursion to sort data
public void mergeSortData();

// Above is group functions
////////////////////////////////////
// Below onwards is individual created functions
// Author: Ethel
// Update data based on it index
public void updateData(T data, int index);

// Author: Ethel
// Swap object based on it index
public void swap(int index1, int index2);

// Author: Khiew Jit Chen
// Return data T and index
public T atIndex(int index);

// Author: Khiew Jit Chen
// Return true if data is equal() the one in the list
public boolean contains(T data);

// Author: Khiew Jit Chen
// Return matched results from 2 list
public SortedListInterface<T> intersect(SortedListInterface<T> otherList);

// Author: Khiew Jit Chen
// Copy data from 1 list to another
public SortedListInterface<T> cloneList();

// Author: Edison Chan Yoong Qi
// Returns a sublist from a main list, filtered by attribute
public SortedListInterface<T> extractByAttributes(SortedListInterface<T> list, String
attribute);

// Author: Edison Chan Yoong Qi
// Returns a new list containing the first N elements
public SortedListInterface<T> getFirstN(int n);

// Author: Edison Chan Yoong Qi
// Returns string list containing unique values from a chosen fieldname
public SortedListInterface<String> extractUniqueValues(String fieldName);

// Author: Edison Chan Yoong Qi

```

```
// Check Duplicates, field_name is the one in ur entity, use that to prevent duplicates
public boolean containsByName(T data, String field_Name);

}
```

1.2.2 Sorted Doubly Linked List

Implementation Class: SortedDoublyLinkedList.java

```
package Adt;

// Author: Edison Chan Yoong Qi, Khiew Jit Chen, Ethel Ng Yi Yan, Hea Hui Min
import java.lang.reflect.Field;
import java.util.Iterator;

public class SortedDoublyLinkedList<T extends Comparable<T>> implements
SortedListInterface<T> {

    private class Node {

        T data;
        Node prev, next;

        Node(T data) {
            this.data = data;
            this.prev = null;
            this.next = null;
        }
    }

    private Node head, tail;
    private int size;

    public SortedDoublyLinkedList() {
        this.head = null;
        this.tail = null;
        this.size = 0;
    }

    @Override
    public int getSize() {
        return size;
    }
}
```

```

@Override
public boolean isEmpty() {
    return size == 0;
}

@Override
public void insertAtHead(T data) {
    Node newNode = new Node(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
    size++;
}

@Override
public void insertAtTail(T data) {
    Node newNode = new Node(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
    size++;
}

@Override
public void insertAtIndex(int index, T data) {
    // Looks like i added 1 too many if statements -Edison
    if (index < 0 || index > size) {
        System.out.println("Invalid index, inserting at head");
        insertAtHead(data);
        return;
    }

    if (index == 0) {
        insertAtHead(data);
        return;
    }

    if (index == size) {
        insertAtTail(data);
        return;
    }

```

```

    }

    Node newNode = new Node(data);
    Node current = head;

    // Traverse to (index - 1)
    for (int i = 0; i < index - 1; i++) {
        current = current.next;
    }

    newNode.next = current.next;
    newNode.prev = current;

    current.next.prev = newNode;
    current.next = newNode;

    size++;
}

@Override
public void deleteFromHead() {
    if (isEmpty()) {
        return;
    }
    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
    size--;
}

@Override
public void deleteFromTail() {
    if (isEmpty()) {
        return;
    }
    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
    size--;
}

@Override

```

```

public void displayForward() {
    Node current = head;

    if (isEmpty()) {
        System.out.println("The list is empty.");
        return;
    }

    while (current != null) {
        System.out.print(current.data);
        if (current.next != null) {
            System.out.println();
        }
        current = current.next;
    }
    System.out.println(); // Move to the next line after printing the list
}

@Override
public void displayBackward() {
    Node current = tail;

    if (isEmpty()) {
        System.out.println("The list is empty.");
        return;
    }

    while (current != null) {
        System.out.print(current.data);
        if (current.prev != null) {
            System.out.println();
        }
        current = current.prev;
    }
    System.out.println(); // Move to the next line after printing the list
}

@Override
public void deleteNode(T data) {
    if (isEmpty()) {
        return;
    }

    Node current = head;

    // Search for the node with the given data
    while (current != null && !current.data.equals(data)) {

```

```

        current = current.next;
    }

    // If node not found
    if (current == null) {
        return;
    }

    // If deleting the head
    if (current == head) {
        deleteFromHead();
    } // If deleting the tail
    else if (current == tail) {
        deleteFromTail();
    } // If deleting from the middle
    else {
        current.prev.next = current.next;
        current.next.prev = current.prev;
        size--;
    }
}

@Override
public Iterator<T> iterator() {
    return new Iterator<T>() {
        private Node current = head; // Start from head

        @Override
        public boolean hasNext() {
            return current != null;
        }

        @Override
        public T next() {
            T data = current.data;
            current = current.next;
            return data;
        }
    };
}

// Special toString()
@Override
public String toString() {
    if (isEmpty()) {
        return "-";
    }
}

```

```

    StringBuilder sb = new StringBuilder("");
    Node current = head; // Assuming 'head' is the starting node
    while (current != null) {
        sb.append(current.data);
        if (current.next != null) {
            sb.append(", ");
        }
        current = current.next;
    }
    sb.append(" ");
    return sb.toString();
}

// Below is the new and improved version, SORTED!!!!
// Inserts element while maintaining sort order
@Override
public void insertSorted(T data) {
    Node newNode = new Node(data);

    if (head == null) {
        head = tail = newNode;
    } else if (data.compareTo(head.data) <= 0) {
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    } else if (data.compareTo(tail.data) >= 0) {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    } else {
        Node current = head;
        while (current != null && data.compareTo(current.data) > 0) {
            current = current.next;
        }
        Node previous = current.prev;

        previous.next = newNode;
        newNode.prev = previous;

        newNode.next = current;
        current.prev = newNode;
    }
    size++;
}

// Merge sort with recursion
@Override
public void mergeSortData() {

```

```

    head = mergeSort(head);
    // Re-adjust tail after sort
    tail = head;
    while (tail != null && tail.next != null) {
        tail = tail.next;
    }
}

// Recursive mergeSort function
private Node mergeSort(Node node) {
    if (node == null || node.next == null) {
        return node;
    }

    Node secondHalf = split(node);

    // Recursively sort both halves
    node = mergeSort(node);
    secondHalf = mergeSort(secondHalf);

    // Merge the sorted halves
    return merge(node, secondHalf);
}

// Split the doubly linked list into two halves
private Node split(Node head) {
    Node slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    Node secondHalf = slow.next;
    slow.next = null;

    if (secondHalf != null) {
        secondHalf.prev = null;
    }

    return secondHalf;
}

// Merge two sorted lists
private Node merge(Node first, Node second) {
    if (first == null) {
        return second;
    }
    if (second == null) {

```

```

        return first;
    }

    if (first.data.compareTo(second.data) <= 0) {
        first.next = merge(first.next, second);
        if (first.next != null) {
            first.next.prev = first;
        }
        first.prev = null;
        return first;
    } else {
        second.next = merge(first, second.next);
        if (second.next != null) {
            second.next.prev = second;
        }
        second.prev = null;
        return second;
    }
}

// Above is group functions
////////////////////////////////////
// Below onwards is individual created functions
//Author : Ethel Ng
@Override
public void updateData(T data, int index) {

    if (index > size || index < 0) {
        System.out.println("Error out of bound");
    }

    Node current = head;
    for (int i = 0; i < index; i++) {
        current = current.next;
    }
    current.data = data;
}

@Override
public void swap(int index1, int index2) {
    if (index1 == index2) {
        return; // No need to swap if indices are the same
    }
    // Ensure indices are within bounds
    if (index1 < 0 || index2 < 0 || index1 >= size || index2 >= size) {
        System.out.println("Error");
        throw new IndexOutOfBoundsException("Invalid index for swap");
    }
}

```

```

// Find the nodes at the given indices
Node node1 = new Node(atIndex(index1));
Node node2 = new Node(atIndex(index2));

updateData(node1.data, index2);
updateData(node2.data, index1);

}

// Author: Khiew Jit Chen
@Override
public T atIndex(int index) {
    if (index < 0 || index >= size) {
        //throw new IndexOutOfBoundsException("Index out of range");
        return null;
    }

    Node current;

    // Optimize: Start from head if index is in the first half, else start from tail
    if (index < size / 2) {
        current = head;
        for (int i = 0; i < index; i++) {
            current = current.next;
        }
    } else {
        current = tail;
        for (int i = size - 1; i > index; i--) {
            current = current.prev;
        }
    }

    return current.data;
}

@Override
public boolean contains(T data) {
    Node current = head;
    while (current != null) {
        if (current.data.equals(data)) {
            return true;
        }
        current = current.next;
    }
    return false;
}

```

```

/**
 * Returns a new sorted list that contains the intersection of this list and
 * the specified other list.
 *
 * @param otherList the other sorted list to compare with
 * @return a sorted list containing elements present in both lists
 * @pre both this list and otherList must be of the same generic type T
 * @pre T must implement Comparable<T>
 */
@Override
public SortedListInterface<T> intersect(SortedListInterface<T> otherList) {
    SortedDoublyLinkedList<T> result = new SortedDoublyLinkedList<>();

    for (T data : this) {
        if (otherList instanceof SortedDoublyLinkedList &&
            ((SortedDoublyLinkedList<T>) otherList).contains(data)) {
            if (!result.contains(data)) {
                result.insertSorted(data);
            }
        }
    }

    return result;
}

@Override
public SortedListInterface<T> cloneList() {
    SortedDoublyLinkedList<T> clone = new SortedDoublyLinkedList<>();
    Node current = head;

    while (current != null) {
        clone.insertSorted(current.data); // assuming you have insertSorted(T data)
        current = current.next;
    }

    return clone;
}

@Override
public SortedListInterface<T> extractByAttributes(SortedListInterface<T> list, String
attribute) {
    SortedListInterface<T> filteredList = new SortedDoublyLinkedList<>();
    Iterator<T> iterator = list.iterator();

    if (list.isEmpty() || attribute.trim().equals("")) {
        return null;
    }
}

```

```

while (iterator.hasNext()) {
    T item = iterator.next();
    // Assuming T has a method to check attributes
    if (matchesAttribute(item, attribute)) {
        filteredList.insertSorted(item);
    }
}

return filteredList;
}

// Helper method to check if an item matches the attribute for the above function
// Had to do some research on this -> my brain slightly bigger than gpt (i used abit for
debugging)
private boolean matchesAttribute(T item, String attribute) {
    // Ensure no empty arguemnts are passed in
    if (item == null || attribute == null) {
        return false;
    }

    // This uses a lang library, for dynamic usage
    try {
        Field[] fields = item.getClass().getDeclaredFields();

        for (Field field : fields) {
            field.setAccessible(true);
            Object value = field.get(item);

            // String
            if (value instanceof String) {
                if (attribute.equalsIgnoreCase((String) value)) {
                    return true;
                }
            } // Numeric
            else if (value instanceof Number) {
                if (attribute.equals(String.valueOf(value))) {
                    return true;
                }
            } // Handle list within a list
            else if (value instanceof SortedListInterface) {
                Iterator<T> iterator = ((SortedListInterface<T>) value).iterator();
                while (iterator.hasNext()) {
                    Object element = iterator.next();
                    if (attribute.equalsIgnoreCase(element.toString())) {
                        return true;
                    }
                }
            }
        }
    }
}

```

```

    }
    } catch (IllegalAccessException e) {
        System.err.println("Error accessing fields: " + e.getMessage());
    }

    return false;
}

@Override
public SortedListInterface<T> getFirstN(int n) {
    SortedDoublyLinkedList<T> result = new SortedDoublyLinkedList<>();
    if (n <= 0 || isEmpty()) {
        return result;
    }

    Node current = head;
    int count = 0;

    while (current != null && count < n) {
        result.insertAtTail(current.data);
        current = current.next;
        count++;
    }

    return result;
}

@Override
public SortedListInterface<String> extractUniqueValues(String fieldName) {
    SortedListInterface<String> uniqueValues = new SortedDoublyLinkedList<>();

    if (isEmpty()) {
        return null;
    }

    // Loops through every item in the given list (this) keyword
    for (T item : this) {
        try {
            Field field = item.getClass().getDeclaredField(fieldName);
            field.setAccessible(true);
            Object value = field.get(item);

            if (value != null) {
                String stringValue = value.toString();
                if (!uniqueValues.contains(stringValue)) {
                    uniqueValues.insertSorted(stringValue);
                }
            }
        }
    }
}

```

```

        } catch (NoSuchFieldException | IllegalAccessException e) {
            System.err.println("Error accessing field '" + fieldName + "': " + e.getMessage());
        }
    }

    return uniqueValues;
}

@Override
public boolean containsByName(T data, String field_Name) {
    if (isEmpty()) {
        return false;
    }

    Node current = head;
    try {
        Field nameFieldRef = data.getClass().getDeclaredField(field_Name);
        nameFieldRef.setAccessible(true);
        String newName = (String) nameFieldRef.get(data);

        while (current != null) {
            String currentName = (String) nameFieldRef.get(current.data);
            if (currentName.equalsIgnoreCase(newName)) {
                return true;
            }
            current = current.next;
        }
    } catch (NoSuchFieldException | IllegalAccessException e) {
        System.err.println("Error accessing name field: " + e.getMessage());
    }
    return false;
}

@Override
public void clear() {
    head = null;
    tail = null;
    size = 0;
}

@Override
public void remove(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException("Index out of bounds: " + index);
    }

    if (index == 0) {

```

```
        deleteFromHead(); // already implemented
        return;
    }

    if (index == size - 1) {
        deleteFromTail(); // already implemented
        return;
    }

    Node current = head;
    for (int i = 0; i < index; i++) {
        current = current.next;
    }

    // Disconnect the node
    current.prev.next = current.next;
    current.next.prev = current.prev;

    size--;
}
}
```

Part B. INDIVIDUAL REPORT

2. Use of ADTs

Name	Student ID	Prog / Tut.Grp	Signature
Edison Chan Yoong Qi	24WMR08859	RDS2 G4	Chan

Module name: Matching Engine

Business Rules

Business Rule:

Matching Engine will only evaluate CGPA (30% weightage), skills (40% weightage), location (20% match), job type preference (10% weightage). Since student expected work experience is near 0, skills all requested by the companies are defaulted to intermediate skill level. Matching Engine is able to be accessed by students to see requirements and trends of companies by seeing the reporting function.

Criteria	Score Calculation
CGPA	(Student CGPA/Job CGPA Requirement) * 0.3
skills	(No. Student Skills/ No. Skills Requirement) * 0.4
location	If same location score is 0.2, else 0.15
job type preference	If job title matches score is 0.1, else 0

Source codes for Control classes

File name: MatchingController.java

```
package Control;

// ADT
import Adt.SortedDoublyLinkedList;
import Adt.SortedListInterface;
// Entity
import Entities.Company;
import Entities.JobPosting;
import Entities.Student;
import Entities.ScoredJob;
// Boundary
import Boundary.InternshipApplicationProgramUI;
import Boundary.StudentUI;
```

```

import Boundary.MatchingUI;
import Entities.Application;
// Utilities
import Utility.MessageUI;
import Utility.UserInput;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

// Author: Edison Chan
public class MatchingController {

    // List from other stuff
    private SortedListInterface<JobPosting> jobPostList = new
SortedDoublyLinkedList<>();
    private SortedListInterface<Student> studentList = new SortedDoublyLinkedList<>();
    private SortedListInterface<Company> companyList = new
SortedDoublyLinkedList<>();
    private SortedListInterface<Application> applicationList = new
SortedDoublyLinkedList<>();
    private SortedListInterface<ScoredJob> scoredJob = new SortedDoublyLinkedList<>();

    // Controllers
    private final JobPostingController JobPostingController = new JobPostingController();
    private final StudentController StudentController = new StudentController();
    private final CompanyController CompanyController = new CompanyController();
    private final ApplicationController ApplicationController =
ApplicationController.getInstance();
    private final MessageUI MessageUI = new MessageUI();

    // Boundaries
    private final MatchingUI MatchingUI = new MatchingUI();
    private final StudentUI StudentUI = new StudentUI();
    private final InternshipApplicationProgramUI InternshipUI = new
InternshipApplicationProgramUI();

    // Utilities
    private final UserInput UserInput = new UserInput();

    // Dependency injection
    // Retrives list from other controllers
    // Just get all at once bruh
    public MatchingController() {
        this.jobPostList = JobPostingController.getJobPostList();
        this.studentList = StudentController.getStudentList();
        this.companyList = CompanyController.getCompanyList();
        this.applicationList = ApplicationController.getApplicationList();
    }

```

```

// Main flow
public void runMatching() {
    int choice = -1;

    InternshipUI.clearScreen();

    // Print main menu
    while (choice != 0) {
        choice = MatchingUI.getMenuChoice();
        switch (choice) {
            case 0: // Exit Program
                break;
            case 1: // search based on company requirement
                initiateSearch();
                break;
            case 2: // search based on user requirement
                customSearch();
                break;
            case 3:
                generateMatchingReport();
            default:
                MessageUI.wrongInput();
                choice = -1;
        }
    }
}

private void initiateSearch() {
    String studentName = MatchingUI.getStudentName();
    Student student = findStudentByName(studentName);

    if (student == null) {
        System.out.println("Student not found!");
        return;
    }

    SortedListInterface<JobPosting> matchedJobs = findBestMatches(student);

    if (matchedJobs.isEmpty()) {
        System.out.println("\nNo suitable jobs found for " + studentName);
    } else {
        displayMatchedJobs(student, matchedJobs);
    }

    StudentUI.enterToContinue();
}

private Student findStudentByName(String name) {

```

```

    for (Student student : studentList) {
        if (student.getName().equalsIgnoreCase(name)) {
            return student;
        }
    }
    return null;
}

private SortedListInterface<JobPosting> findBestMatches(Student student) {
    SortedDoublyLinkedList<ScoredJob> scoredJobs = new
SortedDoublyLinkedList<>();

    // Calculate scores for all matching jobs
    // Do min requirement, filter by CGPA
    for (JobPosting job : jobPostList) {
        if (!(student.getCgpa() < job.getCgpa())) {
            double score = calculateMatchScore(student, job);
            scoredJobs.insertSorted(new ScoredJob(job, score));
        }
    }

    // Extract just the jobs in sorted order
    SortedDoublyLinkedList<JobPosting> matchedJobs = new
SortedDoublyLinkedList<>();
    for (ScoredJob scoredJob : scoredJobs) {
        matchedJobs.insertAtTail(scoredJob.getJob());
    }

    return matchedJobs;
}

private void displayMatchedJobs(Student student, SortedListInterface<JobPosting>
jobs) {
    // Get top 4 matches
    SortedListInterface<JobPosting> top4JobPosting = jobs.getFirstN(4);

    if (top4JobPosting.isEmpty()) {
        System.out.println("No suitable jobs found for " + student.getName());
        return;
    }

    // Convert to array for side-by-side display
    JobPosting[] jobArray = new JobPosting[4];
    int index = 0;
    for (JobPosting job : top4JobPosting) {
        jobArray[index++] = job;
    }
}

```

```

// Display header
System.out.printf("\nTop " + top4JobPosting.getSize() + " Job Matches for " +
student.getName() + ":\n");
System.out.printf("=".repeat(65) + "="+);

if (top4JobPosting.getSize() > 1) {
    System.out.printf("=".repeat(70) + "\n");
} else {
    System.out.println();
}

// Display side by side
System.out.printf("%-65s |", "Match #1");

if (jobArray[1] != null) {
    System.out.printf(" %-65s\n", "Match #2");
} else {
    System.out.println();
}

displayJobPair(jobArray[0], jobArray[1], student);

System.out.printf("-".repeat(65) + "-+");

if (top4JobPosting.getSize() > 1) {
    System.out.printf("-".repeat(70) + "\n");
} else {
    System.out.println();
}

if (jobArray[2] != null) {
    System.out.printf("%-65s |", "Match #3");

    if (jobArray[3] != null) {
        System.out.printf(" %-65s\n", "Match #4");
    } else {
        System.out.println();
    }

    displayJobPair(jobArray[2], jobArray[3], student);
    System.out.printf("-".repeat(65) + "-+" + "-".repeat(70) + "\n");

}
}

private void displayJobPair(JobPosting left, JobPosting right, Student student) {
    String[] leftLines = new String[0];
    String[] rightLines = new String[0];

```

```

        if (left != null) {
            leftLines = getJobDetails(left, student).split("\n");
        }

        if (right != null) {
            rightLines = getJobDetails(right, student).split("\n");
        }

        int maxLines = Math.max(leftLines.length, rightLines.length);

        for (int i = 0; i < maxLines; i++) {
            String rightText = "";
            String leftText = "";

            if (i < leftLines.length) {
                leftText = leftLines[i];
            }
            if (i < rightLines.length) {
                rightText = rightLines[i];
            }

            System.out.printf("%-65s | %-65s\n", leftText, rightText);
        }
    }

    private String getJobDetails(JobPosting job, Student student) {
        if (job == null) {
            return "";
        }
        return String.format(
            " Company: %s\n Company ID: %s\n Position: %s\n Job Description: %s\n\n"
            "Location: %s\n Min CGPA: %s\n Score: %.1f%%",
            job.getCompanyName(),
            job.getID(),
            job.getPosition_title(),
            job.getJob_des(),
            job.getLocation(),
            job.getCgpa(),
            calculateMatchScore(student, job) * 100
        );
    }

    private double calculateMatchScore(Student student, JobPosting job) {
        double score = 0;

        // CGPA match (30% weight)
        if (student.getCgpa() >= job.getCgpa()) {

```

```

        score += 0.3;
    } else {
        score += (student.getCgpa() / job.getCgpa()) * 0.3; // Partial credit
    }

    // Skills match (40% weight)
    int matchedSkills = 0;
    for (String skill : student.getSkills()) {
        if (job.getRequirement().toString().contains(skill)) {
            matchedSkills++;
        }
    }
    if (job.getRequirement().getSize() > 0) {
        score += ((double) matchedSkills / job.getRequirement().getSize()) * 0.4;
    }

    // Location match (20% weight)
    if (student.getLocation().equalsIgnoreCase(job.getLocation())) {
        score += 0.2;
    } else if (job.getCompany().getBranches().contains(student.getLocation())) {
        score += 0.15;
    }

    // Job type preference (10% weight)
    if (student.getJobType().equalsIgnoreCase(job.getPositionType())) {
        score += 0.1;
    }

    return score; // Already capped at 1.0 (100%) by weights
}

////////////////////////////////////
// Search by user own preferences;
private void customSearch() {
    // Colour options
    final String GREENCOLOR = "\u001B[42m";
    final String REDCOLOR = "\u001B[41m";
    final String RESET_BG = "\u001B[0m";

    // Define all possible locations
    String[] locations = {
        "Johor", "Kedah", "Kelantan", "Malacca", "Negeri Sembilan",
        "Pahang", "Penang", "Perak", "Perlis", "Kuala Lumpur",
        "Terengganu", "Sabah", "Sarawak", "Others"
    };

    // ADD HEADER TO SHOW THAT U WANT STUDENT REQUIREMENT

```

```

// Define position types
String[] positionTypes = {"FULL TIME", "PART TIME", "CONTRACT/TEMP"};

// Define salary ranges
String[] salaryRanges = {
    "RM0-800", "RM801-1200", "RM1201-1600", ">RM1600"
};
double[] salaryMinValues = {0, 801, 1201, 1601};
double[] salaryMaxValues = {800, 1200, 1600, Double.MAX_VALUE};

// Extract all the company names from JobListing
SortedListInterface<String> companyNamesList = new SortedDoublyLinkedList<>();
for (JobPosting job : jobPostList) {
    String companyName = job.getCompanyName();
    if (!companyNamesList.contains(companyName)) {
        companyNamesList.insertSorted(companyName);
    }
}

// Track filter states
// Dynamically created to store unique sizes
int[] locationFilterStates = new int[locations.length];
int[] positionFilterStates = new int[positionTypes.length];
int[] salaryFilterStates = new int[salaryRanges.length];
int[] companyFilterStates = new int[companyNamesList.getSize()];

// Current salary range selection, start at -1 cuz no range selected
int selectedSalaryRange = -1;

// Start the search, get user name
String studentName = MatchingUI.getStudentName();
Student student = findStudentByName(studentName);
if (student == null) {
    System.out.println("Student not found!");
    return;
}

// my magnum opus, loops endlessly
// True logic
boolean selectingFilters = true;
while (selectingFilters) {
    InternshipUI.clearScreen();
    System.out.println("CUSTOM JOB MATCH SEARCH - BASED ON YOUR
REQUIREMENTS");
    System.out.println("Custom Search Filters (Press number to toggle, 0 to
finish):\n");

    // Display filters in columns

```

```

        System.out.printf(" %-22s %-27s %-28s %-22s\n", "Salary Ranges", "Location
Filters", "Position Filters", "Company Filters");
        System.out.printf(" %-22s %-27s %-28s %-22s\n", "-----",
"-----", "-----", "-----");

        // Determine maximum rows needed
        // Do this by nesting a shit ton of Math.max to get the true max values
        int maxRows = Math.max(Math.max(salaryRanges.length, locations.length),
Math.max(positionTypes.length, companyNamesList.getSize()));

        // Print everything ROW BY ROW!!!
        for (int i = 0; i < maxRows; i++) {
            // Salary range column
            String salaryText = "";
            if (i < salaryRanges.length) {
                String stateColor = (selectedSalaryRange == i) ? GREENCOLOR + " " +
RESET_BG : " ";
                salaryText = String.format(" S%2d [%s] %-15s", i + 1, stateColor,
salaryRanges[i]);
            }

            // Location column
            String locationText = "";
            if (i < locations.length) {
                String stateColor;
                if (locationFilterStates[i] == 0) {
                    stateColor = " ";
                } else if (locationFilterStates[i] > 0) {
                    stateColor = GREENCOLOR + " " + RESET_BG;
                } else {
                    stateColor = REDCOLOR + " " + RESET_BG;
                }
                locationText = String.format(" L%2d [%s] %-15s", i + 1, stateColor,
locations[i]);
            }

            // Position column
            String positionText = "";
            if (i < positionTypes.length) {
                String stateColor;
                if (positionFilterStates[i] == 0) {
                    stateColor = " ";
                } else if (positionFilterStates[i] > 0) {
                    stateColor = GREENCOLOR + " " + RESET_BG;
                } else {
                    stateColor = REDCOLOR + " " + RESET_BG;
                }
                positionText = String.format(" P%2d [%s] %-20s", i + 1, stateColor,

```

```

positionTypes[i]);
    }

    // Company column
    String companyText = "";
    if (i < companyNamesList.getSize()) {
        String companyName = companyNamesList.atIndex(i);
        String stateColor;
        if (companyFilterStates[i] == 0) {
            stateColor = " ";
        } else if (companyFilterStates[i] > 0) {
            stateColor = GREENCOLOR + " " + RESET_BG;
        } else {
            stateColor = REDCOLOR + " " + RESET_BG;
        }
        companyText = String.format(" C%2d [%s] %-50s", i + 1, stateColor,
companyName);
    }

    System.out.printf("%-25s %-25s %-30s %-30s\n",
        salaryText, locationText, positionText, companyText);
}

System.out.println("\n\n Filter Meanings:");
System.out.printf("[ " + REDCOLOR + " " + RESET_BG + "] = Inactive\t\t" + "[ ]
= Neural\t\t" + "[ " + GREENCOLOR + " " + RESET_BG + "] = Active\t\t");
System.out.println("\n 0  Finish selection");

String choice = UserInput.readString("Enter your choice: ").toUpperCase();

// basically a bunch of else-if statements to reach thru every possible input for filters
// checks the easuiest option first, aka done with filterings
// -1 because you are storing the colour values in an array, in oversimplified terms,
index 1 == index 0
if (choice.equals("0")) {
    selectingFilters = false;
} // Handle salary range selection
else if (choice.startsWith("S") && choice.length() > 1) {
    try {
        int rangeIndex = Integer.parseInt(choice.substring(1)) - 1;
        if (rangeIndex >= 0 && rangeIndex < salaryRanges.length) {
            selectedSalaryRange = (selectedSalaryRange == rangeIndex) ? -1 :
rangeIndex;
        }
    } catch (NumberFormatException e) {
        MessageUI.wrongInput();
    }
} // Handle location filters

```

```

else if (choice.startsWith("L") && choice.length() > 1) {
    try {
        int locIndex = Integer.parseInt(choice.substring(1)) - 1;
        if (locIndex >= 0 && locIndex < locations.length) {
            locationFilterStates[locIndex] = (locationFilterStates[locIndex] + 2) % 3 - 1;
        }
    } catch (NumberFormatException e) {
        MessageUI.wrongInput();
    }
} // Handle position filters
else if (choice.startsWith("P") && choice.length() > 1) {
    try {
        int posIndex = Integer.parseInt(choice.substring(1)) - 1;
        if (posIndex >= 0 && posIndex < positionTypes.length) {
            positionFilterStates[posIndex] = (positionFilterStates[posIndex] + 2) % 3 -
1;
        }
    } catch (NumberFormatException e) {
        MessageUI.wrongInput();
    }
} // Handle company filters
else if (choice.startsWith("C") && choice.length() > 1) {
    try {
        int compIndex = Integer.parseInt(choice.substring(1)) - 1;
        if (compIndex >= 0 && compIndex < companyNamesList.getSize()) {
            companyFilterStates[compIndex] = (companyFilterStates[compIndex] + 2)
% 3 - 1;
        }
    } catch (NumberFormatException e) {
        MessageUI.wrongInput();
    }
} else {
    // If nothing else matters, it does
    MessageUI.wrongInput();
}
// This is basically the end of the while() loop
// If user input 0, begin filter searching
}

System.out.printf("_".repeat(170) + "\n");
// Created a new DLL to store the matchedJobs
// Perform the filtered search and sort by match score
SortedDoublyLinkedList<ScoredJob> matchedJobs = new
SortedDoublyLinkedList<>();

// VERY IMPORTANT FILTER PART
// IF MATCHES -1, WHICH IS THE BLACKLISTED VALUE
// IT IGNORES IT AND CONTINUE TO THE NEXT LIST

```

```

// IF ITS NOT -1, IT WILL ADD INTO A MATCHEDJOB LIST
for (JobPosting job : jobPostList) {
    // Check MINIMUM CGPA first
    if (student.getCgpa() < job.getCgpa()) {
        continue;
    }

    // Then move on to salary filter
    if (selectedSalaryRange != -1) {
        double jobSalary = job.getSalary();
        double minSalary = salaryMinValues[selectedSalaryRange];
        double maxSalary = salaryMaxValues[selectedSalaryRange];

        // Match salary Range
        if (jobSalary < minSalary || jobSalary > maxSalary) {
            continue;
        }
    }
    // Check location filters
    String jobLocation = job.getLocation();
    int locationIndex = -1;
    for (int i = 0; i < locations.length; i++) {
        if (locations[i].equalsIgnoreCase(jobLocation)) {
            locationIndex = i;
            break;
        }
    }
    // Check specifically for "others"
    if (locationIndex == -1) {
        locationIndex = locations.length - 1;
    }

    if (locationFilterStates[locationIndex] < 0) {
        continue;
    }

    boolean hasActiveLocationFilters = false;
    for (int state : locationFilterStates) {
        if (state > 0) {
            hasActiveLocationFilters = true;
            break;
        }
    }
    if (hasActiveLocationFilters && locationFilterStates[locationIndex] <= 0) {
        continue;
    }

    // Check position type filters, parttime, fulltime, contract

```

```

String jobPositionType = job.getPositionType();
int positionIndex = -1;
for (int i = 0; i < positionTypes.length; i++) {
    if (positionTypes[i].equalsIgnoreCase(jobPositionType)) {
        positionIndex = i;
        break;
    }
}

// Check validaity idk how spelll
if (positionIndex != -1 && positionFilterStates[positionIndex] < 0) {
    continue;
}

boolean hasActivePositionFilters = false;
for (int state : positionFilterStates) {
    if (state > 0) {
        hasActivePositionFilters = true;
        break;
    }
}
if (hasActivePositionFilters && (positionIndex == -1 ||
positionFilterStates[positionIndex] <= 0)) {
    continue;
}

// Check company filters
// Helper function below for
String jobCompany = job.getCompanyName();
int companyIndex = getCompanyIndex(companyNamesList, jobCompany);

if (companyIndex != -1 && companyFilterStates[companyIndex] < 0) {
    continue;
}

boolean hasActiveCompanyFilters = false;
for (int state : companyFilterStates) {
    if (state > 0) {
        hasActiveCompanyFilters = true;
        break;
    }
}
if (hasActiveCompanyFilters && (companyIndex == -1 ||
companyFilterStates[companyIndex] <= 0)) {
    continue;
}

// Calculate match score and add to matchedJobs list

```

```

        double score = calculateMatchScore(student, job);
        matchedJobs.insertSorted(new ScoredJob(job, score));

    } // End of the endless loop checks

    // Convert matchedJobs to regular JobPosting list (already sorted by score)
    SortedDoublyLinkedList<JobPosting> filteredJobs = new
SortedDoublyLinkedList<>();
    for (ScoredJob scoredJob : matchedJobs) {
        filteredJobs.insertAtTail(scoredJob.getJob());
    }

    // Display results
    if (filteredJobs.isEmpty()) {
        System.out.println("\nNo jobs found matching your filters");
    } else {
        displayMatchedJobs(student, filteredJobs);
    }

    StudentUI.enterToContinue();
}

// Helper Method for function above
// Returns -1 if there is not match, otherwise return index of company name
private int getCompanyIndex(SortedListInterface<String> companyNamesList, String
companyName) {
    int index = 0;
    for (String name : companyNamesList) {
        if (name.equalsIgnoreCase(companyName)) {
            return index;
        }
        index++;
    }
    return -1;
}

////////////////////////////////////
public void generateMatchingReport() {
    // Use ADT for all data storage
    SortedListInterface<String> companyID =
companyList.extractUniqueValues("companyId");
    SortedListInterface<String> companyNameList =
companyList.extractUniqueValues("companyName");
    SortedListInterface<String> companyLocationList =
companyList.extractUniqueValues("location");
    SortedListInterface<Integer> numberOfJobPostings = new
SortedDoublyLinkedList<>();

```

```

// Get number of job postings per company using ADT
for (Company company : companyList) {
    int count = 0;
    for (JobPosting job : jobPostList) {
        if (job.getCompanyName().equals(company.getCompanyName())) {
            count++;
        }
    }
    numberOfJobPostings.insertAtTail(count);
}

InternshipUI.clearScreen();

// Header
System.out.printf("=".repeat(170) + "\n");
System.out.printf("\t".repeat(7));
System.out.println("TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY");
System.out.printf("\t".repeat(9));
System.out.println("MATCHING MODULE SUBSYSTEM\n");
System.out.printf("\t".repeat(9));
System.out.println("SUMMARY OF MATCHING REPORT");
System.out.printf("\t".repeat(8) + "-".repeat(45) + "\n");
System.out.println("Generated at: " + getCurrentDateTime());
System.out.printf("*".repeat(170) + "\n");
System.out.printf("_".repeat(170) + "\n");
System.out.println("TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY HIGHLY CONFIDENTIAL DOCUMENT");
System.out.printf("_".repeat(170) + "\n");

// Company data table
System.out.printf("%-13s | %-35s | %-11s | %-18s | %-20s \n", "Company ID",
"Company Name", "No. Job Posts", "Headquarters", "Applied Applicants");
System.out.printf("-".repeat(170) + "\n");

// Print every company data
for (int i = 0; i < companyList.getSize(); i++) {
    System.out.printf("    %-8s | %-35s |    %-3s    | %-18s | \t%-20s \n",
        companyList.atIndex(i).getCompanyId(),
        companyList.atIndex(i).getCompanyName(),
        numberOfJobPostings.atIndex(i),
        companyList.atIndex(i).getFullLocation(),
        getNumberOfAppliedStudents(companyList.atIndex(i).getCompanyName()));
}

System.out.printf("_".repeat(170) + "\n");
System.out.println("    Total Number of Companies : " + companyList.getSize());
System.out.println("    Total Number of Job Postings: " + jobPostList.getSize());

```

```

System.out.println();

// Graphical representation
System.out.println("\t".repeat(7) + "GRAPHICAL REPRESENTATION OF
MATCHING MODULE\n");
primaryChart(companyLocationList);

System.out.printf("_".repeat(170) + "\n");

// Footer Data
printJobWithLeastRequirements();
printJobWithMostRequirements();
System.out.println();
System.out.printf("*".repeat(170) + "\n");
System.out.println("\t".repeat(9) + "END OF THE REPORT");
System.out.printf("=".repeat(170) + "\n");

StudentUI.enterToContinue();
}

// Graph 1: Number of job posting to company locations (limit to top 5 locations),
parameters: jobPostList, companyLocationList
// Graph 2: Popular trend for the number of skills (limit to top 5 skills), parameters:
skillList, skillCountList
private void primaryChart(SortedListInterface<String> companyLocationList) {
    final String GREENCOLOR = "\u001B[42m";
    final String RESET_BG = "\u001B[0m";
    final int maxHeight = 10;

    //GRAPH 1: Job Postings per Location (Take only the top 5)
    SortedListInterface<String> locationLabels = new SortedDoublyLinkedList<>();
    SortedListInterface<Integer> locationValues = new SortedDoublyLinkedList<>();

    // Get top 5 locations with most job postings
    // This store both location and job count by location in a single for loop
    for (int i = 0; i < Math.min(5, companyLocationList.getSize()); i++) {
        String location = companyLocationList.atIndex(i);
        locationLabels.insertAtTail(location);
        locationValues.insertAtTail(getJobCountByLocation(location));
    }

    //GRAPH 2: Popular Skills (Take top 5 as well)
    SortedListInterface<String> allSkillsList = new SortedDoublyLinkedList<>();
    SortedListInterface<Integer> skillCounts = new SortedDoublyLinkedList<>();

    // Count skill occurrences across all job postings
    for (JobPosting job : jobPostList) {
        for (String skill : job.getRequirement()) {

```

```

        int index = -1;
        for (int i = 0; i < allSkillsList.getSize(); i++) {
            if (allSkillsList.atIndex(i).equalsIgnoreCase(skill)) {
                index = i;
                break;
            }
        }

        if (index == -1) {
            allSkillsList.insertAtTail(skill);
            skillCounts.insertAtTail(1);
        } else {
            int currentCount = skillCounts.atIndex(index);
            skillCounts.updateData(currentCount + 1, index);
        }
    }
}

// Time to get the top 5 skills
SortedListInterface<String> skillLabels = new SortedDoublyLinkedList<>();
SortedListInterface<Integer> skillValues = new SortedDoublyLinkedList<>();

for (int i = 0; i < 5 && i < allSkillsList.getSize(); i++) {
    int maxIndex = -1;
    int maxValue = -1;

    // Find skill with highest count
    for (int j = 0; j < skillCounts.getSize(); j++) {
        int currentCount = skillCounts.atIndex(j);
        if (currentCount > maxValue) {
            maxValue = currentCount;
            maxIndex = j;
        }
    }

    // If -1 means there is no skills
    if (maxIndex != -1) {
        skillLabels.insertAtTail(allSkillsList.atIndex(maxIndex));
        skillValues.insertAtTail(maxValue);
        skillCounts.updateData(-1, maxIndex); // Mark as processed
    } else {
        skillLabels.insertAtTail("N/A");
        skillValues.insertAtTail(0);
    }
}

// Title
System.out.println("Top Job Postings by Location    " + "t".repeat(5) + "|    Top

```

```

Required Skills");

// Y-axis and bars
System.out.println(" ^" + "\t".repeat(9) + "\t ^");
for (int level = maxHeight; level > 0; level--) {
    System.out.printf("%2d | ", level);

    // First chart (Locations)
    for (int i = 0; i < locationLabels.getSize(); i++) {
        String bar;
        if (locationValues.atIndex(i) >= level) {
            bar = " " + GREENCOLOR + " " + RESET_BG;
        } else {
            bar = " ";
        }
        System.out.printf("%s ", bar);
    }

    // Spacer between charts
    System.out.print("\t\t\t\t\t");
    System.out.printf("%2d | ", level);

    // Second chart (Skills)
    for (int i = 0; i < skillLabels.getSize(); i++) {
        String bar;
        if (skillValues.atIndex(i) >= level) {
            bar = " " + GREENCOLOR + " " + RESET_BG + " ";
        } else {
            bar = " ";
        }
        System.out.printf("%s ", bar);
    }

    System.out.println();
}

// X-axis line
System.out.print(" -+");
for (int i = 0; i < locationLabels.getSize(); i++) {
    System.out.print("-----");
}
System.out.printf("--> Locations \t\t\t -+");
for (int i = 0; i < skillLabels.getSize(); i++) {
    System.out.print("-----");
}
System.out.println("> Skills");

// X-axis labels

```

```

System.out.print("    ");
for (int i = 0; i < locationLabels.getSize(); i++) {
    String label = locationLabels.atIndex(i);
    String trimmed;
    if (label.length() > 4) {
        trimmed = label.substring(0, 5);
    } else {
        trimmed = label;
    }

    System.out.printf("%-8s", trimmed);
}

System.out.print("\t\t\t\t\t");
for (int i = 0; i < skillLabels.getSize(); i++) {
    String label = skillLabels.atIndex(i) + "    ";
    String trimmed;
    if (label.length() > 4) {
        trimmed = label.substring(0, 6);
    } else {
        trimmed = label;
    }
    System.out.printf("%-8s", trimmed.trim());
}

System.out.println();
}

// Helper method for primaryChart()
private int getJobCountByLocation(String location) {
    int count = 0;
    for (JobPosting job : jobPostList) {
        if (job.getLocation().equalsIgnoreCase(location)) {
            count++;
        }
    }
    return count;
}

private void printJobWithLeastRequirements() {
    if (jobPostList == null || jobPostList.getSize() == 0) {
        System.out.println("No job postings in the system.");
        return;
    }
}

// Find the minimum number of requirements
int minRequirements = Integer.MAX_VALUE;

```

```

for (JobPosting job : jobPostList) {
    int reqCount = job.getRequirement().getSize();
    if (reqCount < minRequirements) {
        minRequirements = reqCount;
    }
}

// Collect all jobs with the minimum number of requirements
String result = "";
int jobCount = 0;
for (JobPosting job : jobPostList) {
    if (job.getRequirement().getSize() == minRequirements) {
        if (jobCount > 0) {
            result += ", ";
        }
        result += job.getPosition_title() + " (" + job.getCompanyName() + ")";
        jobCount++;
    }
}

// Print the result
if (jobCount == 1) {
    System.out.println("Job with the least requirements (" + minRequirements + "):\n <
" + result + ">");
} else {
    System.out.println("Jobs with the fewest requirements (" + minRequirements +
    "):\n < " + result + ">");
}

private void printJobWithMostRequirements() {
    JobPosting manyRequirements = new JobPosting();
    if (jobPostList == null || jobPostList.getSize() == 0) {
        System.out.println("No job postings in the system.");
        return;
    }

    // Find the maximum number of requirements
    int maxRequirements = 0;
    for (JobPosting job : jobPostList) {
        int reqCount = job.getRequirement().getSize();
        if (reqCount > maxRequirements) {
            maxRequirements = reqCount;
        }
    }

    // Collect all jobs with the maximum number of requirements
    String result = "";

```

```

int jobCount = 0;
for (JobPosting job : jobPostList) {
    if (job.getRequirement().getSize() == maxRequirements) {
        if (jobCount > 0) {
            result += ", ";
        }
        result += job.getPosition_title() + " (" + job.getCompanyName() + ")";
        manyRequirements = job;
        jobCount++;
    }
}

// Print the result
if (jobCount == 1) {
    System.out.printf("Job with the most requirements (" + maxRequirements + "):\n <
" + result + " > with requirements of:");
    System.out.printf("%s", manyRequirements.getRequirement());
} else {
    System.out.println("Jobs with the most requirements (" + maxRequirements + "):\n
< " + result + " >");
}
}

private int getNumberOfAppliedStudents(String companyName) {
    SortedDoublyLinkedList<String> studentIds = new SortedDoublyLinkedList<>();

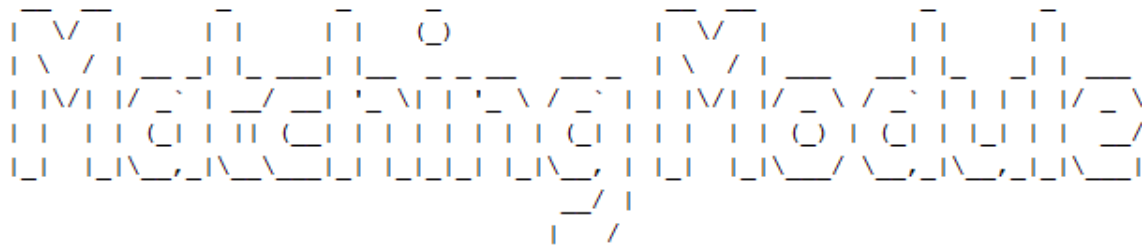
    // Iterate through all applications
    for (Application application : applicationList) {
        // Check if application is for the specified company
        if (application.getJobPosting().getCompanyName().equals(companyName)) {
            String studentId = application.getId();
            // Only count each student once
            if (!studentIds.contains(studentId)) {
                studentIds.insertSorted(studentId);
            }
        }
    }

    return studentIds.getSize();
}

private String getCurrentDateTime() {
    LocalDateTime now = LocalDateTime.now();
    DateTimeFormatter format = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");
    return now.format(format);
}
}

```

Screenshots



```

MATCHING MODULE MAIN MENU
1. Basic Match - Based on Company Requirement
2. Advanced Match - Based on Your Requirement
3. Generate Report
0. Return to Main Menu
Enter choice:

```

Figure 2.1.1 Matching Engine Main Menu

Enter choice: 1

Enter your name : Edison Chan

Top 4 Job Matches for Edison Chan:

Match #1 Company: Microsoft Company ID: JB7 Position: Data Analyst Job Description: Join a dynamic team for data insights Salary: RM 3700.00 Location: Kuala Lumpur Min CGPA: 3.30 Score: 90.0%	Match #2 Company: Otaku Coders Company ID: JB23 Position: Software Developer Job Description: Come be a KING! Salary: RM 2000.00 Location: Kuala Lumpur Min CGPA: 3.40 Score: 76.7%
Match #3 Company: Blue Ocean Company ID: JB25 Position: Software Developer Job Description: Work your life away! Salary: RM 9000.00 Location: Penang Min CGPA: 3.40 Score: 70.0%	Match #4 Company: Amazon Company ID: JB22 Position: Software Developer Job Description: Come be a corporate slave! Salary: RM 400.00 Location: Johor Min CGPA: 3.40 Score: 70.0%

Press ENTER to continue

Figure 2.1.2 Option 1 Basic Match

Basic match just requires the user to input their name and it will return the top 4 results that match the company skill requirements and other factors. Job Matches are displayed according to match score descending.

```

MATCHING MODULE MAIN MENU
1. Basic Match - Based on Company Requirement
2. Advanced Match - Based on Your Requirement
3. Generate Report
0. Return to Main Menu
Enter choice: 2

```

```

Enter your name : Edison Chan

```

Figure 2.1.3 Option 2 first requires the user to input their name to proceed

CUSTOM JOB MATCH SEARCH - BASED ON YOUR REQUIREMENTS

Custom Search Filters (Press number to toggle, 0 to finish):

Salary Ranges	Location Filters	Position Filters	Company Filters
S 1 [] RM0-800	L 1 [] Johor	P 1 [] FULL TIME	C 1 [] Amazon
S 2 [] RM801-1200	L 2 [] Kedah	P 2 [] PART TIME	C 2 [] Ani-Mining
S 3 [] RM1201-1600	L 3 [] Kelantan	P 3 [] CONTRACT/TEMP	C 3 [] Blue Ocean
S 4 [] >RM1600	L 4 [] Malacca		C 4 [] Exploitative Company Z
	L 5 [] Negeri Sembilan		C 5 [] Facebook
	L 6 [] Pahang		C 6 [] Google
	L 7 [] Penang		C 7 [] Intelligent Systems Co.
	L 8 [] Perak		C 8 [] Microsoft
	L 9 [] Perlis		C 9 [] Otaku Coders
	L10 [] Kuala Lumpur		C10 [] Tesla
	L11 [] Terengganu		
	L12 [] Sabah		
	L13 [] Sarawak		
	L14 [] Others		

Filter Meanings:
 [] = Inactive [] = Neural [] = Active
 0 Finish selection
 Enter your choice:

Figure 2.1.4 Filter system of the matching algorithm

User is able to cycle filter state by selecting it multiple times, “0 -> 1 -> -1 -> 0”, 0 indicates no colour, green indicates active, and red indicates blacklist, when the user is done selecting their criteria, they input 0 to finalize their filter

Enter your choice: 0

Top 1 Job Matches for Edison Chan:

=====+

Match #1

Company: Microsoft

Company ID: JB24

Position: Software Developer

Job Description: Fair Pay!

Salary: RM 1400.00

Location: Kuala Lumpur

Min CGPA: 3.40

Score: 66.0%

-----+

Press ENTER to continue|

Figure 2.1.5 Entering 0 outputs the top 4 company that fits your filter criteria, sorted by top score descending



Figure 2.1.6 Option 3 displays the matching report

Matching report displays a variety of fields, from number of job postings per company, to their headquarters and number of applied applicants. This shows the dependency of the matching engine to other modules.

Graphical reports are to show applying students the general trend of skills and general grouping job postings by locations to let them have a better grasp of the job market.

Name	Student ID	Prog / Tut.Grp	Signature
Khiew Jit Chen	24WMR08869	RDS2G4	<i>Khiew</i>

Module name: Application Module

Business Rules for Application Management

1. Unique Applications

Each application must be associated with a specific job posting and applicant (student). Duplicate applications—where the same applicant applies to the same job more than once—are strictly prohibited to prevent spamming.

2. Application Persistence

Applications are persistent records. They cannot be removed arbitrarily, but their status and associated logs can be updated as part of the review and evaluation process.

3. Log Activity and Status Updates

Whenever an application's status is modified or a reviewer adds a comment, the corresponding log activity is updated automatically to reflect the change.

4. Cascade Deletion

If an application is deleted (e.g., due to invalid data or system rules), any interview linked to that application will be cancelled and removed to maintain data integrity.

Source codes for Control classes

File name: ApplicationController.java

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
 this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Control;

import Adt.SortedDoublyLinkedList;
import Adt.SortedListInterface;

import Entities.Application;
import Boundary.ApplicationUI;
import Boundary.InternshipApplicationProgramUI;
import Utility.MessageUI;
import Dao.InternshipDAO;
import Entities.Application.ApplicationStatus;
import Entities.JobPosting;

```

```

import Entities.SkillStats;
import Entities.Student;
import Utility.UserInput;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 *
 * @author Jit Chen
 */
public class ApplicationController {

    private static ApplicationController ApplicationController;
    private int id = 0;

    //controller support
    private StudentController StudentControl = StudentController.getInstance();
    private JobPostingController JobPostingControl = JobPostingController.getInstance();
    private InterviewController InterviewControl;

    //ui module
    private static final ApplicationUI applicationUI = new ApplicationUI();
    private static final UserInput userInput = new UserInput();
    private static final MessageUI MessageUI = new MessageUI();
    private static final InternshipApplicationProgramUI InternshipApplicationProgramUI =
new InternshipApplicationProgramUI();

    //list declared
    private SortedListInterface<Application> applicationList = new
SortedListDoublyLinkedList<>();
    private SortedListInterface<Student> studentList = StudentControl.getStudentList();
    private SortedListInterface<JobPosting> jobPostList =
JobPostingControl.getJobPostList();

    //DAO hardcoded data
    private static final InternshipDAO applicationDAO = new InternshipDAO();
    //hardcoded database

    public static synchronized ApplicationController getInstance() {
        if (ApplicationController == null) {
            ApplicationController = new ApplicationController();
        }
        return ApplicationController;
    }

    public ApplicationController() {
        this.applicationList = applicationDAO.initializeApplicationDAO();
        id = applicationList.getSize();
    }

```

```

}

public void runApplication() {
    displayApplicationMenu();
}

// For other controller to use
public SortedListInterface<Application> getApplicationList() {
    return applicationList;
}

public void displayApplicationMenu() {
    int choice = 0;
    InterviewControl = InterviewController.getInstance();

    InternshipApplicationProgramUI.clearScreen();

    do {
        choice = applicationUI.getMenuChoice();
        switch (choice) {
            case 0:
                System.err.println(MessageUI.exit());
                break;
            case 1:
                //add
                addNewApplication();
                displayApplication();
                break;
            case 2:
                //display
                displayApplication();
                break;
            case 3:
                //modify
                modifyApplicationStatus();
                displayApplication();
                break;
            case 4:
                //delete
                deleteApplication(InterviewControl);
                displayApplication();
                break;
            case 5:
                //application report
                generateApplicationReport();
                break;
            case 6:
                //Application Log

```

```

        applicationLogFunction();
        break;
    case 7:
        //filter
        filterApplication();
        break;
    case 8:

        break;
    default:
        System.err.println(MessageUI.wrongInput());
    }
} while (choice != 0);
}

public void addNewApplication() {
    String jobID = "";
    int studID = 0;
    Student seletedStudent = new Student();
    JobPosting seletedJob = new JobPosting();
    boolean validInput = false;

    //1. show all student to let select their character
    StudentControl.displayAllStudents();

    //2. input to select
    while (!validInput) {
        studID = applicationUI.inputStudentID();
        for (Student student : studentList) {
            if (student.getstudID() == studID) {
                seletedStudent = student;
                validInput = true;
                break;
            } else if (0 == studID) {
                return;
            }
        }
        if (!validInput) {
            System.err.println(MessageUI.wrongInput());
        }
    }

    applicationUI.currentUserInfo(seletedStudent.getName(), seletedStudent.getstudID());

    //3. show job posting to select
    JobPostingControl.displayJobPost();

    //4. input to select

```

```

validInput = false;
while (!validInput) {
    jobID = userInput.inputStringCleaner(applicationUI.inputJobID());
    for (JobPosting job : jobPostList) {
        if (job.getID().equalsIgnoreCase(jobID)) {
            seletedJob = job;
            validInput = true;
            break;
        } else if ("XXX".equalsIgnoreCase(jobID)) {
            return;
        }
    }
    if (!validInput) {
        System.err.println(MessageUI.wrongInput());
    }
}

//5. validation for student dont spam the same job
for (Application application : applicationList) { //same job cannot be apply or the job
is cancelled before
    if (application.getApplicant().equals(seletedStudent) &&
application.getJobPosting().equals(seletedJob) && application.getStatus() !=
ApplicationStatus.CANCELLED) {
        System.err.println(MessageUI.warning() + "You already applied this job!!!!");
        return;
    }
}

//6. create instance and insert to list
Application newApplication = new Application(generateId(++id),seletedJob,
seletedStudent);
applicationList.insertAtTail(newApplication);

}

private String generateId(int id) {
    return String.format("A%04d", id);
}

public String getAllApplication() {
    String outputStr = "";
    for (int i = 0; i < applicationList.getSize(); i++) {

        outputStr += applicationList.atIndex(i).toString();
        if (i != applicationList.getSize() - 1) {
            outputStr += "\n";
        }
    }
}

```

```

    return outputStr;
}

public void displayApplication() {
    applicationUI.listAllApplication(getAllApplication(), applicationList.getSize());
    applicationUI.enterToContinue();
}

public void modifyApplicationStatus() {
    boolean validInput = false;
    int index = 0;
    Application seletedApplication = new Application();
    String appID = "";

    displayApplication();

    //2. input to select
    while (!validInput) {
        appID = applicationUI.inputModifyApplicationID();
        for (Application application : applicationList) {
            if (application.getId().equalsIgnoreCase(appID)) {
                seletedApplication = application;
                validInput = true;
                break;
            } else if ("XXX".equalsIgnoreCase(appID)) {
                return;
            }
            index++;
        }
        if (!validInput) {
            System.err.println(MessageUI.wrongInput());
        }
    }

    modifyOptionPerformed(seletedApplication, index);
}

public void modifyOptionPerformed(Application seletedApplication, int atIndex) {
    int choice = 0;
    do {
        choice = applicationUI.modifyOption();
        switch (choice) {
            case 0 ->
                System.err.println(MessageUI.exit());
            case 1 -> //Reviewed
                seletedApplication.setStatus(ApplicationStatus.REVIEWED);
            case 2 -> //INTERVIEWED

```

```

        seletedApplication.setStatus(ApplicationStatus.INTERVIEWED);
    case 3 -> //ACCEPTED
        seletedApplication.setStatus(ApplicationStatus.ACCEPTED);
    case 4 -> //REJECTED
        seletedApplication.setStatus(ApplicationStatus.REJECTED);
    case 5 -> //CANCELLED
        seletedApplication.setStatus(ApplicationStatus.CANCELLED);
    default ->
        System.err.println(MessageUI.wrongInput());
    }
} while (choice > 5);

applicationList.updateData(seletedApplication, atIndex);
}

public SortedListInterface<String>
dataCollectFromApplicant(SortedListInterface<Student> students) {
    SortedListInterface<String> skillsMap = new SortedDoublyLinkedList<>();

    for (Student student : students) {
        for (String skill : student.getSkills()) {
            skillsMap.insertSorted(skill);
        }
    }

    return skillsMap;
}

public SortedListInterface<String>
dataCollectFromJob(SortedListInterface<JobPosting> jobs) {
    SortedListInterface<String> skillsMap = new SortedDoublyLinkedList<>();

    for (JobPosting job : jobs) {
        for (String skill : job.getRequirement()) {
            skillsMap.insertSorted(skill);
        }
    }

    return skillsMap;
}

public SkillStats dataLoad(SortedListInterface<String> skills) {
    SkillStats skillStats = new SkillStats();

    for (String skill : skills) {
        skillStats.addSkill(skill);
    }
}

```

```

    return skillStats;
}

public SortedListInterface<String> cleanInvalidSkills(SortedListInterface<String>
skills) {

    for (int i = 0; i < skills.getSize(); i++) {
        String skill = skills.atIndex(i);
        if (skill == null || skill.trim().isEmpty() || skill.trim().equals("-") ||
skill.trim().equals("- -")) {
            skills.remove(i); // Don't increment i since list shifts
        } else {
            i++;
        }
    }
    return skills;
}

public SortedListInterface<String> preprocessSkillList(SortedListInterface<String>
rawSkills) {
    SortedListInterface<String> cleanedSkills = new SortedDoublyLinkedList<>(); // Or
your custom class

    for (int i = 0; i < rawSkills.getSize(); i++) {
        String raw = rawSkills.atIndex(i);
        if (raw == null) {
            continue;
        }

        // Step 1: Trim and normalize
        String cleaned = raw.trim().toLowerCase();

        // Step 2: Filter out noisy entries
        if (cleaned.isBlank() || cleaned.length() > 50) {
            continue;
        }

        // Step 3: Convert to title case
        cleaned = toTitleCase(cleaned);

        // Step 4: add
        cleanedSkills.insertAtTail(cleaned);
    }

    return cleanedSkills;
}

```

```

// Helper to convert to Title Case
private String toTitleCase(String input) {
    String[] parts = input.split(" ");
    StringBuilder result = new StringBuilder();

    for (String part : parts) {
        if (!part.isEmpty()) {
            result.append(Character.toUpperCase(part.charAt(0)))
                .append(part.substring(1).toLowerCase())
                .append(" ");
        }
    }

    return result.toString().trim();
}

public void generateApplicationReport() {
    InternshipApplicationProgramUI.clearScreen();
    System.out.println("Generating Report...");

    //collect data from student and job list
    SortedListInterface<String> jobRequirements =
dataCollectFromJob(JobPostingControl.getJobPostList());
    SortedListInterface<String> applicantSkills =
dataCollectFromApplicant(StudentControl.getStudentList());

    //remove invalid entry
    jobRequirements = cleanInvalidSkills(jobRequirements);
    applicantSkills = cleanInvalidSkills(applicantSkills);

    //preprocessing
    jobRequirements = preprocessSkillList(jobRequirements);
    applicantSkills = preprocessSkillList(applicantSkills);

    // data loading to statistic model
    SkillStats jobStats = new SkillStats();
    SkillStats applicantStats = new SkillStats();
    jobStats = dataLoad(jobRequirements);
    applicantStats = dataLoad(applicantSkills);

    applicantStats = applicantStats.getTopNSkills(7);
    jobStats = jobStats.getTopNSkills(7);

    // Place holder
    System.out.printf("=".repeat(170) + "\n");

    System.out.printf("\t".repeat(7));
    System.out.println("TUNKU ABDUL RAHMAN UNIVERSITY OF

```

```

MANAGEMENT AND TECHNOLOGY");

    System.out.printf("\t".repeat(9));
    System.out.println("APPLICATION DATABASE SUBSYSTEM\n");

    System.out.printf("\t".repeat(9));
    System.out.println("Skills Demand vs Availability Report".toUpperCase());
    System.out.printf("\t".repeat(8) + "-".repeat(45) + "\n");

    System.out.println("Generated at: " + getCurrentDateTime());
    System.out.printf("*".repeat(170) + "\n");

    System.out.printf("_".repeat(170) + "\n");
    System.out.println("TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY HIGHLY CONFIDENTIAL DOCUMENT");
    System.out.printf("_".repeat(170) + "\n");
//
//    // TODO:
//    // ADD A COLUMN, NUMBER OF STUDENTS WITH AND WITHOUT AN
//    INTERNSHIP APPLICATION
    System.out.printf("%-10s | %-30s | %-10s || %-10s | %-30s | %-10s \n", "Ranking",
    "Most Popular Job Skills", "Count", "Ranking", "Most Demanded Job Skills", "Count");
    System.out.printf("-".repeat(170) + "\n");
//
    for (int i = 0; i < 7; i++) {
        System.out.printf("    %-6s | %-30s |    %-6s ||    %-6s | %-30s |    %-6s \n", i + 1,
    applicantStats.getSkills().atIndex(i), applicantStats.getCounts().atIndex(i),
        i + 1, jobStats.getSkills().atIndex(i), jobStats.getCounts().atIndex(i));
    }
    System.out.printf("_".repeat(170) + "\n");
    System.out.println("    Total Number of Students    : " + studentList.getSize());
    System.out.println("    Total Number of Job Posting    : " + jobPostList.getSize());
    System.out.println("    Total Number of Application    : " + applicationList.getSize());

    System.out.println();

    System.out.println("\t".repeat(7) + "GRAPHICAL REPRESENTATION OF
STUDENT DISTRIBUTION \n");
    primaryChart(applicantStats, jobStats);

    System.out.printf("_".repeat(170) + "\n");

    System.out.println();

    // Footer data
    System.out.println("The Most Popular Skill is: < " +
    applicantStats.getSkills().atIndex(0) + "> ,total count ( " +
    applicantStats.getCounts().atIndex(0) + ")");

```

```

        System.out.println("The Most Demanded Skill is: < " + jobStats.getSkills().atIndex(0)
+ "> ,total count (" + jobStats.getCounts().atIndex(0) + ")");

        System.out.printf("*".repeat(170) + "\n");
        System.out.println("\t".repeat(9) + "END OF THE REPORT");
        System.out.printf("=".repeat(170) + "\n");

    }

    public String getCurrentDateTime() {
        LocalDateTime now = LocalDateTime.now();
        DateTimeFormatter format = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");
        return now.format(format);
    }

    public void primaryChart(SkillStats applicantSkill, SkillStats jobSkill) {
        int maxHeight = 50 / 3;
        int columnWidth = 8;
        final String COLOR = "\u001B[46m";
        final String RESET_BG = "\u001B[0m";

        // Education Level Labels
        SortedListInterface<String> labels1 = applicantSkill.getSkills();
        SortedListInterface<Integer> values1 = applicantSkill.getCounts();

        // Faculty Lables
        SortedListInterface<String> labels2 = jobSkill.getSkills();
        SortedListInterface<Integer> values2 = jobSkill.getCounts();

        // Title
        System.out.println("Total Count (*3)" + "\t".repeat(8) + "|  Total Count (*3)");

        // Y-axis and bars
        System.out.println("  ^" + "\t".repeat(10) + "\t  ^");
        for (int level = maxHeight; level > 0; level--) {
            System.out.printf("%2d | ", level);

            // First chart
            for (int value : values1) {
                String bar;
                if (value / 3 >= level) {
                    bar = " " + COLOR + "  " + RESET_BG + "";
                } else {
                    bar = "  ";
                }
                System.out.printf("%s ", bar);
            }
        }
    }

```

```

// Spacer between charts
System.out.print("\t\t\t\t");

System.out.printf("%2d | ", level);
// Second chart
for (int value : values2) {
    String bar;
    if (value / 3 >= level) {
        bar = " " + COLOR + "  " + RESET_BG + " ";
    } else {
        bar = "    ";
    }
    System.out.printf("%s ", bar);
}

System.out.println(); // end of this chart row
}

// X-axis line
System.out.print(" -+");
for (int i = 0; i < values1.getSize(); i++) {
    System.out.print("-----");
}
System.out.printf("> Popular Skill \t\t -");

System.out.print("+");
for (int i = 0; i < values2.getSize(); i++) {
    System.out.print("-----");
}
System.out.println("> Demanded Skill");

// X-axis labels
System.out.print("    ");
for (String label : labels1) {
    String trimmed;
    if (label.length() > 4) {
        trimmed = label.substring(0, 4);
    } else {
        trimmed = label;
    }
    System.out.printf("%-" + columnWidth + "s", trimmed);
}

System.out.print("\t\t\t\t");
for (String label : labels2) {
    String trimmed;
    if (label.length() > 4) {

```

```

        trimmed = label.substring(0, 4);
    } else {
        trimmed = label;
    }
    System.out.printf("%-" + columnWidth + "s", trimmed);
}

System.out.println();
}

public void deleteApplication(InterviewController interviewControl) {
    boolean validInput = false;
    String appID = "";
    Application seletedApplication = new Application();

    displayApplication();

    //2. input to select
    while (!validInput) {
        appID = applicationUI.inputDeleteApplicationID();
        for (Application application : applicationList) {
            if (application.getId().equalsIgnoreCase(appID)) {
                seletedApplication = application;
                validInput = true;
                break;
            } else if ("XXX".equalsIgnoreCase(appID)) {
                return;
            }
        }
        if (!validInput) {
            System.err.println(MessageUI.wrongInput());
        }
    }

    //confirmation
    if (userInput.confirmation("Are you sure want to delete this application? (Y/N)")) {
        applicationList.deleteNode(seletedApplication);
        System.out.println("Deleted successfully.");
    } else {
        System.out.println("No delete operation is performed");
    }

    //check application involved in interview
    boolean deleteInterview =
interviewControl.deleteInterviewFromApplication(seletedApplication);
    if(deleteInterview){
        System.err.println("Interview for " + seletedApplication.getId() + " had been
cancelled");
    }
}

```

```

    }

}

public void applicationLogFunction() {
    boolean validInput = false;
    int index = 0;
    Application seletedApplication = new Application();
    String appID = "";

    displayApplication();

    //2. input to select
    while (!validInput) {
        appID = applicationUI.inputApplicationIDForLog();
        for (Application application : applicationList) {
            if (application.getId().equalsIgnoreCase(appID)) {
                seletedApplication = application;
                validInput = true;
                break;
            } else if ("XXX".equalsIgnoreCase(appID)) {
                return;
            }
        }
        index++;
    }
    if (!validInput) {
        System.err.println(MessageUI.wrongInput());
    }
}

logOptionPerformed(seletedApplication, index);

}

public void logOptionPerformed(Application seletedApplication, int index) {
    int choice = 0;
    String keyword = "";
    do {
        choice = applicationUI.logOption();
        switch (choice) {
            case 0:
                System.err.println(MessageUI.exit());
                break;
            case 1:
                //view top log
                int no = applicationUI.inputNo();
                applicationUI.logDisplay("Latest " + no + " Log Views:",
seletedApplication.getLatestLogs(no));

```

```

        break;
    case 2:
        //display all log
        applicationUI.logDisplay("All Activity Log here:",
seletedApplication.logsAllShow());
        break;
    case 3:
        //add comment to application
        addComment(seletedApplication, index);
        applicationUI.logDisplay("All Activity Log here:",
seletedApplication.logsAllShow());
        break;
    case 4:
        //filter by keyword
        keyword = applicationUI.inputKeyword();
        applicationUI.logDisplay("Keyword Matched Logs:",
seletedApplication.logFilterby(keyword));
        break;

    default:
        System.err.println(MessageUI.wrongInput());
    }
} while (choice != 0);
}

public void addComment(Application seletedApplication, int index) {
    String type = "Comment";
    String message = "";

    message = applicationUI.inputComment();
    if (message.equalsIgnoreCase("XXX")) {
        return;
    }
    seletedApplication.logsUpdate(type, message);
    applicationList.updateData(seletedApplication, index);
}

public void filterApplication() {
    SortedListInterface<Application> filteredApplicationList = new
SortedDoublyLinkedList<>();
    String keyword = "";

    keyword = applicationUI.inputKeyword();
    for (Application application : applicationList){
        if(application.toString().toUpperCase().contains(keyword.toUpperCase())){
            filteredApplicationList.insertAtTail(application);
        }
    }
}

```

```
}

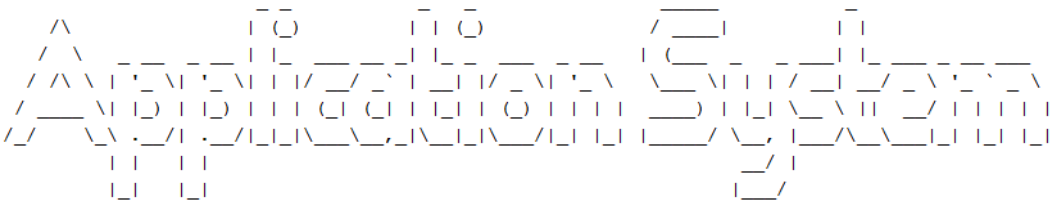
    applicationUI.listAllApplication(filteredApplicationList.toString(),
filteredApplicationList.getSize());

}

}
```

Screenshots

Menu



1. Create new application
2. Display all application
3. Update application
4. Delete application
5. Application Report
6. Application Log
7. Filter application
0. Quit
Enter choice:

Figure 2.2.1 Application Management Main Menu

Add Application

Before add:

App ID	Job ID	Job Title	Company	Stud ID	Stud Name	Matched Skills	Submitted on	Status	Latent Log
A0001	JB10	Frontend Developer	Facebook	2401035	Aileen Jordan	-	18-04-2025 19:06:54	PENDING	<Status> [18-04-2025 19:06:54] this applic
A0002	JB11	Site Reliability Engineer	Google	2401021	Alexander	-	18-04-2025 19:06:54	PENDING	<Status> [18-04-2025 19:06:54] this applic
A0003	JB12	Embedded Systems Engineer	Wells	2401025	Alison Alvarado	C	18-04-2025 19:06:54	PENDING	<Status> [18-04-2025 19:06:54] this applic
A0004	JB13	Devops Engineer	Google	2401026	Anis Shelton	-	18-04-2025 19:06:54	PENDING	<Status> [18-04-2025 19:06:54] this applic
A0005	JB14	QA Tester	Amazon	2401017	Austrian Painter	-	18-04-2025 19:06:54	PENDING	<Status> [18-04-2025 19:06:54] this applic
A0006	JB15	Frontend Developer	Facebook	2401041	Barton Underwood	-	18-04-2025 19:06:54	PENDING	<Status> [18-04-2025 19:06:54] this applic

Total Number of Applications: 6
Press ENTER to continue

After add:

Figure 2.2.2 Application Display before Add

Who are u? Press Q to exit
Enter student ID: 2401002

Current user(Student): Khiew Jit Chen
Student ID: 2401002

J Job ID	J Company Name	J Location	J Position Title	J Job Description	J Job Responsibilities	J Requir
J 2B1	J Google	J Kelantan	J Software Engineer	J Having fun and cozy area for young and mature Engineer to gather and work as a team	J Develop and maintain existing SW	J Node
J 2B2	J Google	J Kelantan	J Junior Software Engineer	J Having fun and cozy area for young and mature Engineer to gather and work as a team	J Develop and maintain existing SW	J Node
J 2B3	J Microsoft	J Kuala Lumpur	J Junior Software Engineer	J Having fun and cozy area for young and mature Engineer to gather and work as a team	J Develop and maintain existing SW	J ASP.+
J 2B4	J Amazon	J Johor	J Junior Software Engineer	J Having fun and cozy area for young and mature Engineer to gather and work as a team	J Develop and maintain existing SW	J ASP.+
J 2B5	J Twila	J Megeri Sembilan	J Junior Software Engineer	J Having fun and cozy area for young and mature Engineer to gather and work as a team	J Develop and maintain existing SW	J ASP.+
J 2B6	J Google	J Kelantan	J Backend Developer	J Work on scalable backend services	J Build API and manage databases	J 3r+.
J 2B7	J Microsoft	J Kuala Lumpur	J Data Analyst	J Join a dynamic team for data insights	J Analyze and report key trends	J Excel
J 2B8	J Amazon	J Johor	J Cloud Engineer	J Innovative cloud solutions	J Work with AWS and microservices	J AWS
J 2B9	J Twila	J Megeri Sembilan	J AI Engineer	J Push boundaries of autonomous tech	J Implement ML models in production	J Python
J 2B10	J Facebook	J Kedah	J Frontend Developer	J Create engaging UIs	J React, HTML, CSS, JavaScript	J HTML
J 2B11	J Google	J Kelantan	J Site Reliability Engineer	J Maintain uptime and resilience	J Monitoring, CI/CD, cloud ops	J Linux
J 2B12	J Twila	J Megeri Sembilan	J Embedded Systems Engineer	J Launch with us into the future	J Write embedded C for avionics	J C. 80
J 2B13	J Google	J Kelantan	J DevOps Engineer	J Fast-paced CI/CD pipelines	J Build robust deployment pipelines	J Docker
J 2B14	J Amazon	J Johor	J QA Tester	J Ensure product quality	J Write and automate test cases	J Selenium
J 2B15	J Microsoft	J Kuala Lumpur	J Security Analyst	J Defend systems from threats	J Security monitoring and response	J SIEM
J 2B16	J Amazon	J Johor	J Data Engineer	J Fix and develop databases	J Fix and develop databases	J POCB
J 2B17	J Intelligent Systems Co.	J Johor	J Software Engineer	J Fix and develop software applications	J Fix and develop software applications	J POCB
J 2B18	J Anti-Mining	J Kelantan	J Database Engineer	J Develop and fix databases	J Develop and fix databases	J POCB
J 2B19	J Otaku Coders	J Kuala Lumpur	J Database Engineer	J Develop and fix databases	J Develop and fix databases	J POCB
J 2B20	J Otaku Coders	J Kuala Lumpur	J Software Tester	J Create unit tests	J Create unit test for applications	J Java
J 2B21	J Amazon	J Johor	J Cloud Developer	J Develop and fix cloud networks	J Develop and maintain existing Software	J AWS
J 2B22	J Amazon	J Johor	J Software Developer	J Come be a corporate slave!	J Develop and maintain existing Software	J Java
J 2B23	J Otaku Coders	J Kuala Lumpur	J Software Developer	J Come be a HNS!	J Develop and maintain existing Software	J Java
J 2B24	J Microsoft	J Kuala Lumpur	J Software Developer	J Pair Pug!	J Develop and maintain existing Software	J Java
J 2B25	J Blue Ocean	J Penang	J Software Developer	J Work your life away!	J Develop and maintain existing Software	J Java
J 2B26	J Exploitative Company 2	J Kuala Lumpur	J Overworked Intern	J Be a slave forever!	J Develop and maintain existing Software	J Rust
J 2B27	J Exploitative Company 2	J Kuala Lumpur	J Software Engineer	J Be a slave forever!	J Develop and maintain existing Software	J Rust
J 2B28	J Exploitative Company 2	J Kuala Lumpur	J Data Analyst	J Be a slave forever!	J Analyze data	J Rust
J 2B29	J Exploitative Company 2	J Kuala Lumpur	J Quality Assurance	J Be a slave forever!	J Assure Quality	J Rust

Which job u like to apply? Press XXX to quit.
Enter JobID: jh1

List of Application:

J App ID	J Job ID	J Job Title	J Company	J Stud ID	J Stud Name	J Matched Skills	J Submitted on	J Status	J Latest Log
J A0001	J 2B10	J Frontend Developer	J Facebook	J 2401035	J Aileen Jordan	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0002	J 2B11	J Site Reliability Engineer	J Google	J 2401021	J Alexander	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0003	J 2B12	J Embedded Systems Engineer	J Twila	J 2401025	J Aline Alvarado	J C	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0004	J 2B13	J DevOps Engineer	J Google	J 2401026	J Amie Shelton	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0005	J 2B14	J QA Tester	J Amazon	J 2401017	J Austrian Painter	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0006	J 2B10	J Frontend Developer	J Facebook	J 2401041	J Barton Underwood	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0007	J 2B1	J Software Engineer	J Google	J 2401002	J Khiew Jit Chen	J Java	J 18-04-2025 19:08:21	J PENDING	J <Status> [18-04-2025 19:08:21] This applica

Total Number of Application: 7
Press ENTER to continue

Figure 2.2.2 Application Display After Add

This function is performed to create applications for applicants to apply for jobs. It does not allow the spam application to the same job from the same applicant.

Display Application

List of Application:

J App ID	J Job ID	J Job Title	J Company	J Stud ID	J Stud Name	J Matched Skills	J Submitted on	J Status	J Latest Log
J A0001	J 2B10	J Frontend Developer	J Facebook	J 2401035	J Aileen Jordan	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0002	J 2B11	J Site Reliability Engineer	J Google	J 2401021	J Alexander	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0003	J 2B12	J Embedded Systems Engineer	J Twila	J 2401025	J Aline Alvarado	J C	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0004	J 2B13	J DevOps Engineer	J Google	J 2401026	J Amie Shelton	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0005	J 2B14	J QA Tester	J Amazon	J 2401017	J Austrian Painter	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0006	J 2B10	J Frontend Developer	J Facebook	J 2401041	J Barton Underwood	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica

Total Number of Application: 6
Press ENTER to continue

Figure 2.2.3 Application List Display

This function is performed to display the whole applications list.

Modify Application

Before:

List of Application:

J App ID	J Job ID	J Job Title	J Company	J Stud ID	J Stud Name	J Matched Skills	J Submitted on	J Status	J Latest Log
J A0001	J 2B10	J Frontend Developer	J Facebook	J 2401035	J Aileen Jordan	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0002	J 2B11	J Site Reliability Engineer	J Google	J 2401021	J Alexander	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0003	J 2B12	J Embedded Systems Engineer	J Twila	J 2401025	J Aline Alvarado	J C	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0004	J 2B13	J DevOps Engineer	J Google	J 2401026	J Amie Shelton	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0005	J 2B14	J QA Tester	J Amazon	J 2401017	J Austrian Painter	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica
J A0006	J 2B10	J Frontend Developer	J Facebook	J 2401041	J Barton Underwood	J -	J 18-04-2025 19:06:54	J PENDING	J <Status> [18-04-2025 19:06:54] This applica

Total Number of Application: 6
Press ENTER to continue

J Status	J Latest Log
J PENDING	J <STATUS> [18-04-2025 19:19:09] This application is submitted by Aileen Jordan
J PENDING	J <STATUS> [18-04-2025 19:19:09] This application is submitted by Alexander
J PENDING	J <STATUS> [18-04-2025 19:19:09] This application is submitted by Aline Alvarado
J PENDING	J <STATUS> [18-04-2025 19:19:09] This application is submitted by Amie Shelton
J PENDING	J <STATUS> [18-04-2025 19:19:09] This application is submitted by Austrian Painter
J PENDING	J <STATUS> [18-04-2025 19:19:09] This application is submitted by Barton Underwood

Figure 2.2.4 Application List Display Before Modify

After:

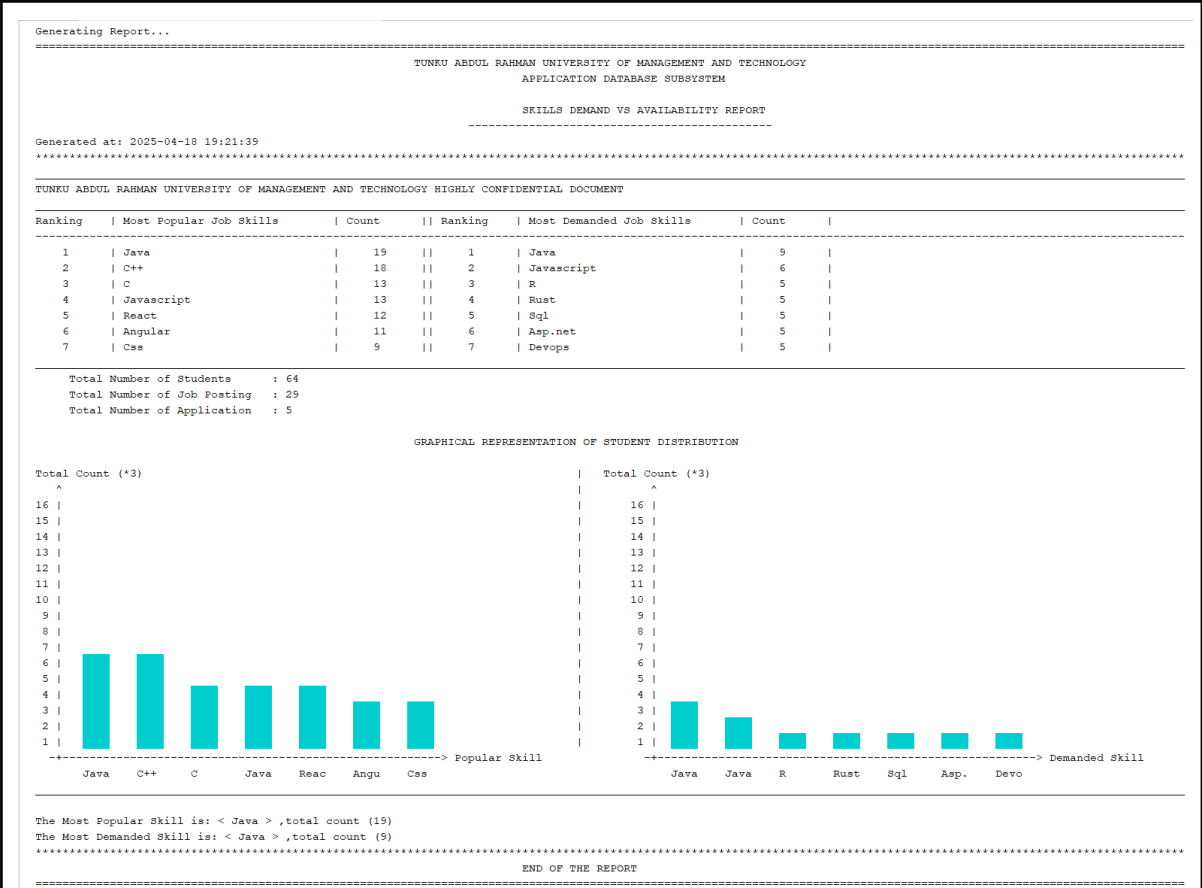


Figure 2.2.8 Application Report

This function is performed to show a simple report including graphical and table view that perform descriptive analysis about the popular skills in applicants and high demand skills in job posting.

Display log

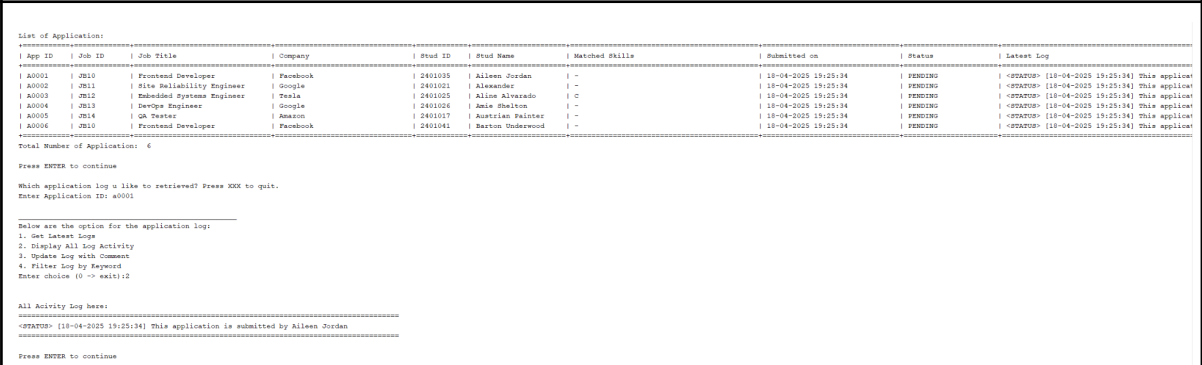


Figure 2.2.9 Application Log Display

This function is performed to show the update history and comments from the reviewer of the application.

Update Log with comment

Before:

All Activity Log here:
=====

<STATUS> [18-04-2025 19:25:34] This application is submitted by Aileen Jordan
=====

Figure 2.2.10 Application Log Display before Comment

After:

Below are the option for the application log:

1. Get Latest Logs
2. Display All Log Activity
3. Update Log with Comment
4. Filter Log by Keyword
Enter choice (0 -> exit):3

Comment by who? Press XXX to quit.
Enter Name: Lionel
Enter Message: Nice and pro applicant

All Activity Log here:
=====

<COMMENT> [18-04-2025 19:27:54] Nice and pro applicant by Lionel
<STATUS> [18-04-2025 19:25:34] This application is submitted by Aileen Jordan
=====

Press ENTER to continue

Figure 2.2.11 Application Log Display after Comment

This function is performed to let the reviewer to comment on the application.

Filter log with keyword

Filter by Keyword
Enter Keyword: comment

Keyword Matched Logs:
=====

<COMMENT> [18-04-2025 19:34:08] Nice and pro applicant by Lionel
=====

Press ENTER to continue

Figure 2.2.11 Application Log Display Filter with Keyword

This function is performed to filter the log activity by keyword on the application.

Filter application with keyword

Filter by Keyword
Enter Keyword: a0001

List of Application:

App ID	Job ID	Job Title	Company	Stud ID	Stud Name	Matched Skills	Submitted on	Status	Latest Log
A0001	J010	Frontend Developer	Facebook	2401035	Aileen Jordan	-	18-04-2025 19:53:37	PENDING	<STATUS> [18-04-2025 19:53:37] This applicat

Total Number of Applications: 1

```
Filter by Keyword
Enter Keyword: a0001

List of Application:
+=====+=====+=====+
| App ID   | Job ID   | Job Title
+=====+=====+=====+
| A0001    | JB10     | Frontend Developer
+=====+=====+=====+

Total Number of Application:  1
```

jkl

Figure 2.2.12 Application Display Filter with Keyword

This function is performed to filter the application by using keyword.

Name	Student ID	Prog / Tut.Grp	Signature
Ethel Ng Yi Yan	24WMR08861	RDSG4	Ethel

Module name: Job Management Module

Business Rules for Job Management Module

1. Job Posting Format

- Job IDs are automatically assigned to every job post to ensure uniqueness in the format: JBx, where x refers to the sequential number of the job post.
- Job IDs may lead to duplicate IDs if initialised manually, which should be avoided or handled properly.

2. Employment Type

- Companies using the internship program system are not limited to hired interns only.
- They may also use the system to hire part time or full time positions

3. Minimum Salary Requirement

- The salary or allowance specified in any job post must not be lower than 200.

1. Source codes for Control classes

Filename : JobPostingController.java

```
package Control;

/**
 *
 * @author Ethel
 */
//Adt
import Adt.SortedDoublyLinkedList;
import Adt.SortedListInterface;

//Boundary
import Boundary.JobPostingUI;

//Utility
import Utility.UserInput;
import Utility.MessageUI;

//Entity
import Entities.JobPosting;
```

```

//Dao
import Dao.InternshipDAO;

//Others
import java.util.Scanner;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class JobPostingController {

    private static JobPostingController JobPostingController;

    // List Declaration
    SortedListInterface<JobPosting> jobPostList = new SortedDoublyLinkedList<>();

    //Controller
    private CompanyController CompanyController;

    //Utility
    private static UserInput UserInput;

    //Boundary
    private static JobPostingUI JobPostingUI;

    // Dao - hardcoded data
    private final InternshipDAO JobPostingDAO = new InternshipDAO();

    // Entity
    private static final JobPosting JobPosting = new JobPosting();

    public final Scanner scanner = new Scanner(System.in);
    private int id = 0;

    public JobPostingController() {
        jobPostList = JobPostingDAO.initializeJobPostingDAO();
        JobPostingUI = new JobPostingUI();
        id = jobPostList.getSize();
    }

    public SortedListInterface<JobPosting> getJobPostList() {
        return jobPostList;
    }

    public static synchronized JobPostingController getInstance() {
        if (JobPostingController == null) {
            JobPostingController = new JobPostingController();
        }
    }
}

```

```

    return JobPostingController;
}

//done
public void mainJobPost() {
    int option;
    this.CompanyController = CompanyController.getInstance();
    reloadCompany();
    do {
        option = JobPostingUI.getJobPostMenuChoice();

        if (option >= 0 && option <= 8) {
            switch (option) {
                case 1 -> {

                    displayJobPost();

                    do {
                        addJobPost();
                    } while (JobPostingUI.continueMessage());

                    displayJobPost();
                }
                case 2 -> {
                    // Search and Filter
                    int choice;
                    do {
                        choice = JobPostingUI.searchFilterOption();

                        if (choice == 1) {
                            displayJobPost();
                            System.out.println();
                            searchJobPost();

                            if (!JobPostingUI.continueMessage()) {
                                break;
                            }
                        }

                        } else if (choice == 2) {
                            filterJobPost();

                            if (!JobPostingUI.continueMessage()) {
                                break;
                            }
                        }

                        } else if (choice == 0) {
                            MessageUI.exit();
                            break;
                        }
                    }
                }
            }
        }
    }
}

```

```

        } else {
            MessageUI.wrongInput();
        }

    } while (true);

}
case 3 -> {
    // Display Job Post
    do {
        displayJobPost();
    } while (JobPostingUI.continueMessage());
}
case 4 -> {
    // Edit Job Post
    boolean yes;
    do {

        displayJobPost();
        editJobPost();

        yes = JobPostingUI.continueMessage();
        if (yes == false) {
            displayJobPost();
        }
    } while (yes);
}
case 5 -> {
    // Delete Job Post
    do {
        displayJobPost();
        System.out.println("");
        deleteJobPost();
        System.out.println("");
        displayJobPost();

    } while (JobPostingUI.continueMessage());
}
case 6 -> {
    // Delete All Job Posts
    displayJobPost();
    do {
        deleteAllJobPost();
    } while (JobPostingUI.continueMessage());
    displayJobPost();
}
case 7 -> {

```

```

        // Sort Job Posts
        do {
            sortJobPost();
        } while (JobPostingUI.continueMessage());
    }
    case 8 -> {
        // Report Job Posts
        do {
            reportJobPost();
        } while (JobPostingUI.continueMessage());
    }
    case 0 -> {
        System.out.println("Exiting Job Posting Menu...");
    }
}
} else {
    MessageUI.wrongInput();
}

} while (option != 0);
}

//Generate ID
public String genID() {
    id++;
    return "JB" + (id);
}

public int insertJobPostBack(JobPosting jobP) {
    int size = jobPostList.getSize();
    jobPostList.insertAtTail(jobP);
    if (jobPostList.getSize() == size) {
        return 1; //Error occur
    }
    return 0; // no Error
}

public void addJobPost() {
    JobPosting input = new JobPosting();
    String id = JobPostingUI.getJobPostCompanyId();

    if (input.getCompanyById(id, CompanyController) == 1) { // company does not Exist
in the COMPANY LIST
        return;
    }

    input.setPosition_title(JobPostingUI.getJobPositionTitle());

```

```

int jobPosition;
do {
    jobPosition = JobPostingUI.getJobPosition(input.getAllPosition());
} while (input.setPositionIndex(jobPosition) == 1);

double salary;
do {
    salary = JobPostingUI.getJobPostingSalary();
} while (input.checkMinSalary(salary) == 1);

input.setJob_des(JobPostingUI.getJobPostingDescription());
input.setJob_respon(JobPostingUI.getJobPostingResp());

do {
    input.addRequirement(JobPostingUI.getJobReq());
} while (JobPostingUI.addMoreReq().equalsIgnoreCase("y"));
double cgpa;
do {
    cgpa = JobPostingUI.getCgpa();
} while (input.checkCGPA(cgpa) == 1);

String id_ = this.genID();
input.setID(id_);
this.insertJobPostBack(input);
JobPostingUI.printLatestID(id_);
}

public int getJobPostById(String id) {

    for (int i = 0; i < jobPostList.getSize(); i++) {
        if (jobPostList.atIndex(i).getID().equals(id)) {
            return i;
        }
    }
    return -1;
}

public void searchJobPost() {
    String id = JobPostingUI.printSearchID();
    int index = getJobPostById(id);
    if (index == -1) {
        System.out.println("Error unable to found Job Post");
        return;
    }
    jobPostList.atIndex(index).displayJobPost();
}

```

```

//done
public void displayJobPost() {
    JobPostingUI.printDisplayHead();
    this.sortByJobId();
    jobPostList.displayForward();
    JobPostingUI.printLine();
}

public void editJobPost() {
    int option;
    String id = JobPostingUI.printSearchID();
    int index = getJobPostById(id);
    if (index == -1) {
        System.out.println("Error unable to found Job Post");
        return;
    }
    JobPosting found = jobPostList.atIndex(index);
    found.displayJobPost();
    System.out.println("");

    do {
        option = JobPostingUI.editOption();
        if (option >= 0 && option <= 7) {
            switch (option) {

                case 1 -> {
                    String title = JobPostingUI.getJobPositionTitle();
                    found.setPosition_title(title);
                    found.displayJobPost();
                }
                case 2 -> {
                    //Edit Position
                    int jobPosition;
                    do {
                        jobPosition = JobPostingUI.getJobPosition(found.getAllPosition());
                    } while (found.setPositionIndex(jobPosition) == 1);
                    found.displayJobPost();
                }
                case 3 -> {
                    //Edit Salary Offer
                    double salary;
                    do {
                        salary = JobPostingUI.getJobPostingSalary();

                    } while (found.checkMinSalary(salary) == 1);
                    found.displayJobPost();
                }
                case 4 -> {

```

```

        //Edit Job Description
        found.setJob_des(JobPostingUI.getJobPostingDescription());
        found.displayJobPost();

    }
    case 5 -> {
        //Edit Job Responsibility
        found.setJob_respon(JobPostingUI.getJobPostingResp());
        found.displayJobPost();

    }
    case 6 -> {
        //Edit Job Requirement
        found.removeAllRequirement();
        do {
            found.addRequirement(JobPostingUI.getJobReq());
        } while (JobPostingUI.addMoreReq().equalsIgnoreCase("y"));
        found.displayJobPost();

    }
    case 7 -> {
        // Edit Min. CGPA
        double cgpa;
        do {
            cgpa = JobPostingUI.getCgpa();
        } while (found.checkCGPA(cgpa) == 1);
        found.displayJobPost();

    }
    case 0 -> {
        MessageUI.exit();
    }
}
} else {
    MessageUI.wrongInput();
}

} while (option < 0 || option > 7);

}

public void deleteAllCompanyName(String name) {

    for (int i = 0; i < jobPostList.getSize(); i++) {
        JobPosting temp = jobPostList.atIndex(i);
        if (temp.getCompanyName().equalsIgnoreCase(name)) {
            jobPostList.deleteNode(temp);
        }
    }
}

```



```

        this.sortBySalary();
        if (desc == 1) {
            JobPostingUI.printDisplayHead();
            jobPostList.displayForward();
        } else if (desc == 2) {
            JobPostingUI.printDisplayHead();
            jobPostList.displayBackward();
        }
    } while (desc != 1 && desc != 2);
}
case 2 -> {

    int desc;

    displayJobPost();

    do {
        desc = JobPostingUI.sortAscDes();
        this.sortByJobId();
        if (desc == 1) {
            JobPostingUI.printDisplayHead();
            jobPostList.displayForward();
        } else if (desc == 2) {
            JobPostingUI.printDisplayHead();
            jobPostList.displayBackward();
        }
    } while (desc != 1 && desc != 2);
}
case 3 -> {
    int desc;

    displayJobPost();

    do {
        desc = JobPostingUI.sortAscDes();
        this.sortByCompanyName();
        if (desc == 1) {
            JobPostingUI.printDisplayHead();
            jobPostList.displayForward();
        } else if (desc == 2) {
            JobPostingUI.printDisplayHead();
            jobPostList.displayBackward();
        }
    } while (desc != 1 && desc != 2);

}
case 4 -> {
    int desc;

```

```

        displayJobPost();

        do {
            desc = JobPostingUI.sortAscDes();
            this.sortByLocation();
            if (desc == 1) {
                JobPostingUI.printDisplayHead();
                jobPostList.displayForward();
            } else if (desc == 2) {
                JobPostingUI.printDisplayHead();
                jobPostList.displayBackward();
            }
        } while (desc != 1 && desc != 2);

    }
    case 0 -> {
        MessageUI.exit();
    }

}
JobPostingUI.printLine();
break;
} else {
    MessageUI.wrongInput();
}

} while (option != 0);

}

public void filterJobPost() {
    int option;

    do {

        option = JobPostingUI.filterOption();
        if (option >= 0 && option <= 2) {

            switch (option) {

                case 1 -> {
                    displayJobPost();
                    filterJobPostByCompany();
                }

                case 2 -> {

```

```

        displayJobPost();
        filterJobPostBySalary();

    }
    case 0 -> {
        MessageUI.exit();
    }

}
} else {
    MessageUI.wrongInput();
}

} while (option != 0);
}

public void reportJobPost() {
    int option;

    do {
        option = JobPostingUI.reportOption();

        if (option == 1) {
            JobPostingUI.reportHeader1();
            System.out.println("Generated by " + getCurrentDateTime());
            JobPostingUI.printDoubleLine();
            displayReportJobPost();
            JobPostingUI.printDoubleLine();
            graphNoCompanyHiring();
            JobPostingUI.printDoubleLine();
            displayTotalJobPost();
            JobPostingUI.printDoubleLine();

            break;
        } else if (option == 0) {
            MessageUI.exit();
        } else {
            MessageUI.wrongInput();
        }

    } while (option != 0);
}

// Generate date time for report
public static String getCurrentDateTime() {
    LocalDateTime now = LocalDateTime.now();
    DateTimeFormatter format = DateTimeFormatter.ofPattern("yyyy-MM-dd

```

```

HH:mm:ss");
    return now.format(format);
}

public String countCompanyInList(String count[], String uniqueName) {
    int result = 0;

    for (String s : count) {
        if (s.equalsIgnoreCase(uniqueName)) {
            result++;
        }
    }

    return uniqueName + ":" + result;
}

public void graphNoCompanyHiring() {
    String result = "";
    String CompanyName = "";

    // Collect all company name and entries
    for (int i = 0; i < jobPostList.getSize(); i++) {
        result += jobPostList.atIndex(i).getCompanyName() + ",";
        if (!CompanyName.contains(jobPostList.atIndex(i).getCompanyName())) {
            CompanyName += jobPostList.atIndex(i).getCompanyName() + ",";
        }
    }

    String count[] = result.split(",", -1);
    CompanyName = CompanyName.substring(0, CompanyName.length() - 1);
    String uniqueName[] = CompanyName.split(",");

    result = ""; //reset result
    for (String s : uniqueName) {
        result += countCompanyInList(count, s) + ",";
    }
    result = result.substring(0, result.length() - 1);
    String temp[] = result.split(",", -1);

    System.out.println("Company : Job Posting Count");
    for (String s : temp) {
        String noComp[] = s.split(":");
        System.out.printf("%s : %s\n", noComp[0], noComp[1]);
    }
    System.out.println("\n\n");

    genBarChart(temp);
}

```

```

}

public void sortBySalary() {
    boolean sort = false; // no sorting occur

    for (int i = 0; i < jobPostList.getSize(); i++) {

        for (int y = 0; y < jobPostList.getSize() - 1; y++) {

            JobPosting current = (JobPosting) jobPostList.atIndex(y);
            JobPosting next = (JobPosting) jobPostList.atIndex(y + 1);
            try {
                if (current.getSalary() > next.getSalary()) {
                    jobPostList.swap(y, y + 1);
                    sort = true; // sorting occur
                }
            } catch (Exception e) {
                System.out.println(e.toString());
            }
        }

        if (sort == false) {
            break; // sorting
        }
        sort = false; //reset sort
    }
}

public void sortByJobId() {

    boolean sort = false; // no sorting occur

    for (int i = 0; i < jobPostList.getSize(); i++) {

        for (int y = 0; y < jobPostList.getSize() - 1; y++) {
            JobPosting current = (JobPosting) jobPostList.atIndex(y);
            JobPosting next = (JobPosting) jobPostList.atIndex(y + 1);
            if (current.getIdValue() > next.getIdValue()) {
                jobPostList.swap(y, y + 1);
                sort = true; // sorting occur
            }
        }

        if (sort == false) {
            break; // sorting
        }
    }
}

```

```

        sort = false; //reset sort

    }

}

public void sortByCompanyName() {
    boolean sort = false;
    for (int i = 0; i < jobPostList.getSize(); i++) {

        for (int y = 0; y < jobPostList.getSize() - 1; y++) {
            JobPosting current = (JobPosting) jobPostList.atIndex(y);
            JobPosting next = (JobPosting) jobPostList.atIndex(y + 1);
            if (current.getCompanyName().compareTo(next.getCompanyName()) > 0) {
                jobPostList.swap(y, y + 1);
                sort = true; // sorting occur
            }
        }

        if (sort == false) {
            break; // sorting
        }
        sort = false; //reset sort

    }

}

public void sortByLocation() {
    boolean sort = false;
    for (int i = 0; i < jobPostList.getSize(); i++) {

        for (int y = 0; y < jobPostList.getSize() - 1; y++) {
            JobPosting current = (JobPosting) jobPostList.atIndex(y);
            JobPosting next = (JobPosting) jobPostList.atIndex(y + 1);
            if (current.getLocation().compareTo(next.getLocation()) > 0) {
                jobPostList.swap(y, y + 1);
                sort = true; // sorting occur
            }
        }

        if (sort == false) {
            break; // sorting
        }
        sort = false; //reset sort

    }

}

```

```

public void filterJobPostByCompany() {
    String company = JobPostingUI.getJobPostCompanyName();
    boolean found = false;

    // Table Header
    JobPostingUI.printDisplayHead();
    for (int i = 0; i < jobPostList.getSize(); i++) {
        JobPosting job = jobPostList.atIndex(i); // assuming this returns JobPosting

        if (job.getCompanyName().equalsIgnoreCase(company)) {
            job.jobPostDisplayFormatted();
            found = true;
        }
    }

    JobPostingUI.printDoubleLine();

    if (!found) {
        System.out.println("No job posts found for company: " + company);
    }
}

public void filterJobPostBySalary() {
    double minSalary = JobPostingUI.getMinSalary(); // method to get minimum salary
    from user
    double maxSalary = JobPostingUI.getMaxSalary(); // method to get maximum salary
    from user
    boolean found = false;

    // Table Header
    JobPostingUI.printDisplayHead();
    for (int i = 0; i < jobPostList.getSize(); i++) {
        JobPosting job = jobPostList.atIndex(i); // assuming this returns JobPosting

        if (job.getSalary() >= minSalary && job.getSalary() <= maxSalary) {
            job.jobPostDisplayFormatted();
            found = true;
        }
    }

    JobPostingUI.printDoubleLine();

    if (!found) {
        System.out.println("No job posts found in salary range: " + minSalary + " - " +
maxSalary);
    }
}

```

```

public void genBarChart(String data[]) {
    String[] labels = new String[data.length];
    int[] values = new int[data.length];
    int maxHeight = 0;

    // Parse input data
    for (int i = 0; i < data.length; i++) {
        String[] parts = data[i].split(":");
        labels[i] = parts[0];
        values[i] = Integer.parseInt(parts[1]);
        if (values[i] > maxHeight) {
            maxHeight = values[i];
        }
    }

    System.out.println("  ^");

    // Print vertical bars with Y-axis
    for (int i = maxHeight; i > 0; i--) {
        System.out.printf("%02d | ", i); // Y-axis label
        for (int j = 0; j < values.length; j++) {
            if (values[j] >= i) {
                System.out.printf("%-25s", "*");
            } else {
                System.out.printf("%-25s", " ");
            }
        }
        System.out.println();
    }

    // Print X-axis
    System.out.print("  +");
    for (int i = 0; i < data.length; i++) {
        System.out.print("-----");
    }

    System.out.println(">");
    System.out.println();

    // Print labels
    System.out.print("    ");
    for (int i = 0; i < labels.length; i++) {
        System.out.printf("%-25s", labels[i]);
    }
    System.out.println();
}

```

```

public void displayTotalJobPost(){
    int indexLarge = 0;
    int valueLarge = Integer.MIN_VALUE;
    String display = "";
    int indexSmall = 0;
    int valueSmall = Integer.MAX_VALUE;

    for(int i = 0; i < CompanyController.getCompanyList().getSize(); i++){
        int count = 0;
        String currentCompanyName =
CompanyController.getCompanyList().atIndex(i).getCompanyName();

        for(int j = 0; j < jobPostList.getSize(); j++){
            if(jobPostList.atIndex(j).getCompanyName().equals(currentCompanyName))
                count++;
        }
        if(count >= valueLarge){
            valueLarge = count;
            indexLarge = i;
        }
        if(count != 0)
            if(count <= valueSmall){
                valueSmall = count;
                indexSmall = i;
            }
        display+=String.format("%-10s : %d,",currentCompanyName, count);
    }

    System.out.printf("\n");
    System.out.printf("Company with largest Job Posting is \n");
    String temp[] = display.split(",");
    for(String s : temp){
        if(s.contains(""+valueLarge))
            System.out.println(s);
    }
    System.out.printf("\n");

    System.out.printf("Company with smallest Job Posting is \n");
    temp = display.split(",");
    for(String s : temp){
        if(s.contains(""+valueSmall))
            System.out.println(s);
    }
}
}

```

```

public void displayReportJobPost(){
    JobPostingUI.printReportDisplayHead();
    this.sortByJobId();
    for (int i = 0; i < jobPostList.getSize(); i++) {
        JobPosting job = jobPostList.atIndex(i); // assuming this returns JobPosting
        job.jobPostReportFormat();

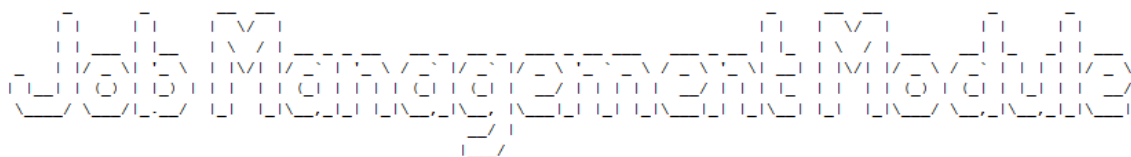
    }
    JobPostingUI.printLine();
}

private void reloadCompany(){
    for(int i = 0; i < CompanyController.getCompanyList().getSize(); i++){
        for(int j = 0; j < jobPostList.getSize(); j++) {
            String JbCmpId = jobPostList.atIndex(j).getCompany().getCompanyId();
            String CmpId =
CompanyController.getCompanyList().atIndex(i).getCompanyId();
            if(JbCmpId.equals(CmpId)){

jobPostList.atIndex(j).setCompany(CompanyController.getCompanyList().atIndex(i));
            }
        }
    }
}
}

```

2. Screenshots



```

Job posting Main Menu
1. Add New Job Posting
2. Search or Filter Job Posting
3. Display All Job Posting
4. Edit Job Posting Information
5. Delete Job Posting
6. Delete All Job Posting
7. Sort Job Posting
8. Job Posting Report
0. Quit
-----
Enter choice (0-8): |

```

Diagram of Display Job Management Main Menu

Enter position title : Software Developer

Result :

Job Post ID : JB1

Job Company Name : Google

Job Location : Kelantan

Job Title : Software Developer

Job Type: FULL TIME

Salary offer : RM 3000.00

Job Requirement :

1. Node.js
2. Java
3. SpringBoot
4. ASP.net
5. Flutter
6. Devops

Minimum CGPA : 4.00

Job Responsible : Develop and maintain existing SW

Job Description : Having fun and cozy area for young and mature Engineer to gather and work as a team

Do you want to continue? (Y = Yes, N = No): N

Diagram 4.3 : Diagram of Edit Job Post Position Title and Display the Information of the Specific Job Post

Diagram 4.4 : Diagram of Display All Job Posting After Edit the Specific Job Post

Option 2 : Edit Position Type

Enter Salary offer : 4000

Result :

Job Post ID : JB1

Job Company Name : Google

Job Location : Kelantan

Job Title : Software Engineer

Job Type: FULL TIME

Salary offer : RM 4000.00

Job Requirement :

1. Node.js
2. Java
3. SpringBoot
4. ASP.net
5. Flutter
6. Devops

Minimum CGPA : 4.00

Job Responsible : Develop and maintain existing SW

Job Description : Having fun and cozy area for young and mature Engineer to gather and work as a team

Do you want to continue? (Y = Yes, N = No): N

Diagram 4.7 : Diagram of Edit Job Post Salary and Display the Information of the Specific Job Post

Diagram 4.8 : Diagram of Display All Job Posting After Edit the Specific Job Post

Option 4 : Edit Description

Enter job description : Maintain update and resilient

Result :

Job Post ID : JB1

Job Company Name : Google

Job Location : Kelantan

Job Title : Software Engineer

Job Type: FULL TIME

Salary offer : RM 3000.00

Job Requirement :

1. Node.js
2. Java
3. SpringBoot
4. ASP.net
5. Flutter
6. Devops

Minimum CGPA : 4.00

Job Responsible : Develop and maintain existing SW

Job Description : Maintain update and resilient

```
Do you want to continue? (Y = Yes, N = No): n
```

Diagram 4.9 : Diagram of Edit Job Post Description and Display the Information of the Specific Job Post


```
Enter job requirement : Java
Any more requirement Y to continue :Y
Enter job requirement : Python
Any more requirement Y to continue :Y
Enter job requirement : C++
Any more requirement Y to continue :N
```

Result :

Job Post ID : JB1
Job Company Name : Google
Job Location : Kelantan
Job Title : Software Engineer
Job Type: FULL TIME
Salary offer : RM 3000.00
Job Requirement :
 1. Java
 2. Python
 3. C++
Minimum CGPA : 4.00
Job Responsible : Develop and maintain existing SW
Job Description : Having fun and cozy area for young and mature Engineer to gather and work as a team

Do you want to continue? (Y = Yes, N = No): N

Diagram 4.12 : Diagram of Edit Job Post Requirement and Display the Information of the Specific Job Post

[illegible]

Diagram 4.13 : Diagram of Display All Job Posting After Edit the Specific Job Post

Option 7 : Edit CGPA Requirement

Enter Minimum CGPA Requirement: 2.4

Result :

```

Job Post ID : JB1
Job Company Name : Google
Job Location : Kelantan
Job Title : Software Engineer
Job Type: FULL TIME
Salary offer : RM 3000.00
Job Requirement :
    1. Node.js
    2. Java
    3. SpringBoot
    4. ASP.net
    5. Flutter
    6. Devops
Minimum CGPA : 2.40
Job Responsible : Develop and maintain existing SW
Job Description : Having fun and cozy area for young and mature Engineer to gather and work as a team
-----
Do you want to continue? (Y = Yes, N = No): N

```

Diagram 4.14 : Diagram of Edit Job Post Minimum CGPA Requirement and Display the Information of the Specific Job Post

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	525	526	527	528	529	530	531	532	533	534	535	536	537	538	539	540	541	542	543	544	545	546	547	548	549	550	551	552	553	554	555	556	557	558	559	560	561	562	563	564	565	566	567	568	569	570	571	572	573	574	575	576	577	578	579	580	581	582	583	584	585	586	587	588	589	590	591	592	593	594	595	596	597	598	599	600	601	602	603	604	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620	621	622	623	624	625	626	627	628	629	630	631	632	633	634	635	636	637	638	639	640	641	642	643	644	645	646	647	648	649	650	651	652	653	654	655	656	657	658	659	660	661	662	663	664	665	666	667	668	669	670	671	672	673	674	675	676	677	678	679	680	681	682	683	684	685	686	687	688	689	690	691	692	693	694	695	696	697	698	699	700	701	702	703	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720	721	722	723	724	725	726	727	728	729	730	731	732	733	734	735	736	737	738	739	740	741	742	743	744	745	746	747	748	749	750	751	752	753	754	755	756	757	758	759	760	761	762	763	764	765	766	767	768	769	770	771	772	773	774	775	776	777	778	779	780	781	782	783	784	785	786	787	788	789	790	791	792	793	794	795	796	797	798	799	800	801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820	821	822	823	824	825	826	827	828	829	830	831	832	833	834	835	836	837	838	839	840	841	842	843	844	845	846	847	848	849	850	851	852	853	854	855	856	857	858	859	860	861	862	863	864	865	866	867	868	869	870	871	872	873	874	875	876	877	878	879	880	881	882	883	884	885	886	887	888	889	890	891	892	893	894	895	896	897	898	899	900	901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920	921	922	923	924	925	926	927	928	929	930	931	932	933	934	935	936	937	938	939	940	941	942	943	944	945	946	947	948	949	950	951	952	953	954	955	956	957	958	959	960	961	962	963	964	965	966	967	968	969	970	971	972	973	974	975	976	977	978	979	980	981	982	983	984	985	986	987	988	989	990	991	992	993	994	995	996	997	998	999	1000
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------

Diagram 4.4 : Diagram of Display All Job Posting After Edit the Specific Job Pos

The Edit Job Posting Function in the Job Management Module allows users to amend a specific job post by entering the job post ID. The function will display the information of the specific job post to acknowledge the user once the job post ID is provided. It allows users to update on job title, job type, salary offer, job requirement, and the minimum CGPA requirement by choosing a different option from the Edit Job Post Menu. This function not only ensures the job posting information is accurate and up-to-date, therefore it can also allow fault tolerance such as typo mistakes made by users.

5.0 Search and Filter Function :

Search or Filter Function

1. Search Function

2. Filter Function

0. Exit

Option :

Diagram 5.0 : Diagram of Search or Filter Menu

Option 1: Search Function

Diagram 5.1 : Diagram of Display All Job Posting Before Search Job Pos

Filter Function Option 2 : Filter Job Posting By Salary

Diagram 5.6 : Diagram of Display All Job Posting Before Filter By Specific Criteria

Diagram 5.7 : Diagram of Display Job Posting After Filter by Salary Range

```
Enter minimum salary: 1000
Enter maximum salary: 2000
```

Requirements	Salary	Position	CGPA	Position Type
Node.js ,Java ,SpringBoot ,ASP.net ,Flutter ,Devops	2000.00	2	4.00	CONTRACT/TEMP
SQL , AWS , PostGre , MySQL , Oracle	1200.00	0	3.40	FULL TIME
Java , JavaScript , R	2000.00	0	3.40	FULL TIME
Java , JavaScript , R , R++ , Communication 101	1400.00	0	3.40	FULL TIME

Diagram 5.7.1 : Diagram of Display Job Posting After Filter by Specific Salary Range (Zoom In)

The Search Job Posting Function in the Job Management Module allows users to search a specific job post after entering the job post ID. It allows users to quickly identify the job post that they wanted to view by displaying the information of the specific job post.

The Filter Job Posting Function in the Job Management Module allows users to search job posts by specific company or salary range. It is useful when it can quickly allocate job opportunities from a preferred company or identify jobs within a salary range.

6.0 Sort Function:

Sort Job Posting

- ```

1. Sort By Salary
2. Sort By Job ID
3. Sort By Company Name
4. Sort By Location
0. Exit

```

Option : 1

*Diagram 6.0 : Diagram of Sort Function Menu*

### Option 1 : Sort By Salary



| Job ID | Company Name | Location          | Position Title    | Job Description                                                   | Job Responsibilities                                              | Requirements                          | Salary    | Position | DOB        | Position Type |
|--------|--------------|-------------------|-------------------|-------------------------------------------------------------------|-------------------------------------------------------------------|---------------------------------------|-----------|----------|------------|---------------|
| 10001  | Google       | Mountain View, CA | Software Engineer | Develop and maintain Google's search algorithms.                  | Develop and maintain Google's search algorithms.                  | BS in Computer Science or equivalent. | \$100,000 | 10       | 10/10/1980 | Full Time     |
| 10002  | Facebook     | Menlo Park, CA    | Software Engineer | Develop and maintain Facebook's social networking features.       | Develop and maintain Facebook's social networking features.       | BS in Computer Science or equivalent. | \$90,000  | 10       | 10/10/1980 | Full Time     |
| 10003  | Amazon       | Seattle, WA       | Software Engineer | Develop and maintain Amazon's e-commerce platform.                | Develop and maintain Amazon's e-commerce platform.                | BS in Computer Science or equivalent. | \$80,000  | 10       | 10/10/1980 | Full Time     |
| 10004  | Microsoft    | Redmond, WA       | Software Engineer | Develop and maintain Microsoft's operating system.                | Develop and maintain Microsoft's operating system.                | BS in Computer Science or equivalent. | \$70,000  | 10       | 10/10/1980 | Full Time     |
| 10005  | Apple        | Cupertino, CA     | Software Engineer | Develop and maintain Apple's mobile devices.                      | Develop and maintain Apple's mobile devices.                      | BS in Computer Science or equivalent. | \$60,000  | 10       | 10/10/1980 | Full Time     |
| 10006  | Netflix      | Los Gatos, CA     | Software Engineer | Develop and maintain Netflix's streaming service.                 | Develop and maintain Netflix's streaming service.                 | BS in Computer Science or equivalent. | \$50,000  | 10       | 10/10/1980 | Full Time     |
| 10007  | Uber         | San Francisco, CA | Software Engineer | Develop and maintain Uber's ride-sharing platform.                | Develop and maintain Uber's ride-sharing platform.                | BS in Computer Science or equivalent. | \$40,000  | 10       | 10/10/1980 | Full Time     |
| 10008  | Twitter      | San Francisco, CA | Software Engineer | Develop and maintain Twitter's social media platform.             | Develop and maintain Twitter's social media platform.             | BS in Computer Science or equivalent. | \$30,000  | 10       | 10/10/1980 | Full Time     |
| 10009  | LinkedIn     | Sunnyvale, CA     | Software Engineer | Develop and maintain LinkedIn's professional networking platform. | Develop and maintain LinkedIn's professional networking platform. | BS in Computer Science or equivalent. | \$20,000  | 10       | 10/10/1980 | Full Time     |
| 10010  | Slack        | San Francisco, CA | Software Engineer | Develop and maintain Slack's team collaboration platform.         | Develop and maintain Slack's team collaboration platform.         | BS in Computer Science or equivalent. | \$10,000  | 10       | 10/10/1980 | Full Time     |

Diagram 6.4 : Diagram of Display All Job Posting After Sorting by Salary with Descending

| Requirements                                        | Salary   |
|-----------------------------------------------------|----------|
| FOCS , SQL , AWS                                    | 10000.00 |
| FOCS , SQL , AWS                                    | 10000.00 |
| FOCS , Java , C++ , JavaScript                      | 10000.00 |
| FOCS , SQL , C , C++ , Rust , COBOL , ASM x86       | 10000.00 |
| Java , JavaScript                                   | 9000.00  |
| C ,RTOS ,hardware integration                       | 8000.00  |
| Python ,TensorFlow ,PyTorch                         | 7000.00  |
| SIEM tools ,network security                        | 6100.00  |
| ASP.net ,MsSQL ,Linux ,Devops                       | 6000.00  |
| Linux ,Docker ,Prometheus                           | 5800.00  |
| AWS ,Docker ,Kubernetes ,Java                       | 5200.00  |
| Jenkins ,Docker ,K8s ,Git                           | 5100.00  |
| 3+ years in Java ,Spring ,MongoDB                   | 4500.00  |
| Minimum 2 years experience in frontend dev          | 4000.00  |
| ASP.net ,MsSQL ,Linux ,Devops                       | 4000.00  |
| Excel ,SQL , Python , HTML , CSS                    | 3700.00  |
| ASP.net ,MsSQL ,Linux ,Devops                       | 3400.00  |
| Selenium ,JUnit ,API testing                        | 3200.00  |
| Node.js ,Java ,SpringBoot ,ASP.net ,Flutter ,Devops | 3000.00  |
| Java , JavaScript , R                               | 2000.00  |
| Node.js ,Java ,SpringBoot ,ASP.net ,Flutter ,Devops | 2000.00  |
| Java , JavaScript , R , R++ , Communication 101     | 1400.00  |
| SQL , AWS , PostGre , MySQL , Oracle                | 1200.00  |
| Java , JavaScript , API_Testing , Firebrain         | 800.00   |
| Java , JavaScript                                   | 400.00   |
| Rust                                                | 50.00    |
| Rust , R , C , C++ , Angle , Angular                | 20.00    |
| Rust , R                                            | 20.00    |
| Rust , R , C                                        | 0.00     |

Diagram 6.4.1 : Diagram of Display All Job Posting After Sorting by Salary with Descending

Option 2 : Sort by Job ID  
Sort Ascending:

| Job ID | Company Name | Location          | Position Title    | Job Description                                                   | Job Responsibilities                                              | Requirements                          | Salary    | Position | DOB        | Position Type |
|--------|--------------|-------------------|-------------------|-------------------------------------------------------------------|-------------------------------------------------------------------|---------------------------------------|-----------|----------|------------|---------------|
| 10001  | Google       | Mountain View, CA | Software Engineer | Develop and maintain Google's search algorithms.                  | Develop and maintain Google's search algorithms.                  | BS in Computer Science or equivalent. | \$100,000 | 10       | 10/10/1980 | Full Time     |
| 10002  | Facebook     | Menlo Park, CA    | Software Engineer | Develop and maintain Facebook's social networking features.       | Develop and maintain Facebook's social networking features.       | BS in Computer Science or equivalent. | \$90,000  | 10       | 10/10/1980 | Full Time     |
| 10003  | Amazon       | Seattle, WA       | Software Engineer | Develop and maintain Amazon's e-commerce platform.                | Develop and maintain Amazon's e-commerce platform.                | BS in Computer Science or equivalent. | \$80,000  | 10       | 10/10/1980 | Full Time     |
| 10004  | Microsoft    | Redmond, WA       | Software Engineer | Develop and maintain Microsoft's operating system.                | Develop and maintain Microsoft's operating system.                | BS in Computer Science or equivalent. | \$70,000  | 10       | 10/10/1980 | Full Time     |
| 10005  | Apple        | Cupertino, CA     | Software Engineer | Develop and maintain Apple's mobile devices.                      | Develop and maintain Apple's mobile devices.                      | BS in Computer Science or equivalent. | \$60,000  | 10       | 10/10/1980 | Full Time     |
| 10006  | Netflix      | Los Gatos, CA     | Software Engineer | Develop and maintain Netflix's streaming service.                 | Develop and maintain Netflix's streaming service.                 | BS in Computer Science or equivalent. | \$50,000  | 10       | 10/10/1980 | Full Time     |
| 10007  | Uber         | San Francisco, CA | Software Engineer | Develop and maintain Uber's ride-sharing platform.                | Develop and maintain Uber's ride-sharing platform.                | BS in Computer Science or equivalent. | \$40,000  | 10       | 10/10/1980 | Full Time     |
| 10008  | Twitter      | San Francisco, CA | Software Engineer | Develop and maintain Twitter's social media platform.             | Develop and maintain Twitter's social media platform.             | BS in Computer Science or equivalent. | \$30,000  | 10       | 10/10/1980 | Full Time     |
| 10009  | LinkedIn     | Sunnyvale, CA     | Software Engineer | Develop and maintain LinkedIn's professional networking platform. | Develop and maintain LinkedIn's professional networking platform. | BS in Computer Science or equivalent. | \$20,000  | 10       | 10/10/1980 | Full Time     |
| 10010  | Slack        | San Francisco, CA | Software Engineer | Develop and maintain Slack's team collaboration platform.         | Develop and maintain Slack's team collaboration platform.         | BS in Computer Science or equivalent. | \$10,000  | 10       | 10/10/1980 | Full Time     |

Diagram 6.5: Diagram of Display All Job Posting After Sorting by Job ID with Ascending

| Job ID | Company Name            | Location        | Position Title            |
|--------|-------------------------|-----------------|---------------------------|
| JB1    | Google                  | Kelantan        | Software Engineer         |
| JB2    | Google                  | Kelantan        | Junior Software Engineer  |
| JB3    | Microsoft               | Kuala Lumpur    | Junior Software Engineer  |
| JB4    | Amazon                  | Johor           | Junior Software Engineer  |
| JB5    | Tesla                   | Negeri Sembilan | Junior Software Engineer  |
| JB6    | Google                  | Kelantan        | Backend Developer         |
| JB7    | Microsoft               | Kuala Lumpur    | Data Analyst              |
| JB8    | Amazon                  | Johor           | Cloud Engineer            |
| JB9    | Tesla                   | Negeri Sembilan | AI Engineer               |
| JB10   | Facebook                | Kedah           | Frontend Developer        |
| JB11   | Google                  | Kelantan        | Site Reliability Engineer |
| JB12   | Tesla                   | Negeri Sembilan | Embedded Systems Engineer |
| JB13   | Google                  | Kelantan        | DevOps Engineer           |
| JB14   | Amazon                  | Johor           | QA Tester                 |
| JB15   | Microsoft               | Kuala Lumpur    | Security Analyst          |
| JB16   | Amazon                  | Johor           | Data Engineer             |
| JB17   | Intelligent Systems Co. | Johor           | Software Engineer         |
| JB18   | Ani-Mining              | Kelantan        | Database Engineer         |
| JB19   | Otaku Coders            | Kuala Lumpur    | Database Engineer         |
| JB20   | Otaku Coders            | Kuala Lumpur    | Software Tester           |
| JB21   | Amazon                  | Johor           | Cloud Developer           |
| JB22   | Amazon                  | Johor           | Software Developer        |
| JB23   | Otaku Coders            | Kuala Lumpur    | Software Developer        |
| JB24   | Microsoft               | Kuala Lumpur    | Software Developer        |
| JB25   | Blue Ocean              | Penang          | Software Developer        |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Overworked Intern         |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Software Engineer         |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Data Analyst              |
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Quality Assurance         |

Diagram 6.5.1 : Diagram of Display All Job Posting After Sorting by Job ID with Ascending (Zoom In)

## Sort Descending:

| Job ID | Company Name            | Location        | Position Title            | Job Description                                           | Job Requirements | Responsibilities                           | Salary   | Experience | EDRS  | Position Type |
|--------|-------------------------|-----------------|---------------------------|-----------------------------------------------------------|------------------|--------------------------------------------|----------|------------|-------|---------------|
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Quality Assurance         | We are a client looking for a QA Tester.                  | Recent Graduate  | Test, report, and track bugs.              | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Data Analyst              | We are a client looking for a Data Analyst.               | Recent Graduate  | Collect, analyze, and visualize data.      | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Software Engineer         | We are a client looking for a Software Engineer.          | Recent Graduate  | Develop and maintain web applications.     | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Overworked Intern         | We are a client looking for an Overworked Intern.         | Recent Graduate  | Assist with various tasks.                 | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB25   | Blue Ocean              | Penang          | Software Developer        | We are a client looking for a Software Developer.         | Recent Graduate  | Develop and maintain web applications.     | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB24   | Microsoft               | Kuala Lumpur    | Software Developer        | We are a client looking for a Software Developer.         | Recent Graduate  | Develop and maintain web applications.     | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB23   | Otaku Coders            | Kuala Lumpur    | Software Developer        | We are a client looking for a Software Developer.         | Recent Graduate  | Develop and maintain web applications.     | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB22   | Amazon                  | Johor           | Cloud Developer           | We are a client looking for a Cloud Developer.            | Recent Graduate  | Develop and maintain cloud applications.   | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB21   | Amazon                  | Johor           | Cloud Developer           | We are a client looking for a Cloud Developer.            | Recent Graduate  | Develop and maintain cloud applications.   | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB20   | Otaku Coders            | Kuala Lumpur    | Software Tester           | We are a client looking for a Software Tester.            | Recent Graduate  | Test and report bugs.                      | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB19   | Otaku Coders            | Kuala Lumpur    | Database Engineer         | We are a client looking for a Database Engineer.          | Recent Graduate  | Design and maintain databases.             | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB18   | Ani-Mining              | Kelantan        | Database Engineer         | We are a client looking for a Database Engineer.          | Recent Graduate  | Design and maintain databases.             | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB17   | Intelligent Systems Co. | Johor           | Software Engineer         | We are a client looking for a Software Engineer.          | Recent Graduate  | Develop and maintain web applications.     | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB16   | Amazon                  | Johor           | Data Engineer             | We are a client looking for a Data Engineer.              | Recent Graduate  | Collect, analyze, and visualize data.      | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB15   | Microsoft               | Kuala Lumpur    | Security Analyst          | We are a client looking for a Security Analyst.           | Recent Graduate  | Monitor and analyze security threats.      | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB14   | Amazon                  | Johor           | QA Tester                 | We are a client looking for a QA Tester.                  | Recent Graduate  | Test, report, and track bugs.              | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB13   | Google                  | Kelantan        | DevOps Engineer           | We are a client looking for a DevOps Engineer.            | Recent Graduate  | Manage infrastructure and deployments.     | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB12   | Tesla                   | Negeri Sembilan | Embedded Systems Engineer | We are a client looking for an Embedded Systems Engineer. | Recent Graduate  | Develop and test embedded systems.         | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB11   | Google                  | Kelantan        | Site Reliability Engineer | We are a client looking for a Site Reliability Engineer.  | Recent Graduate  | Monitor and maintain system uptime.        | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB10   | Facebook                | Kedah           | Frontend Developer        | We are a client looking for a Frontend Developer.         | Recent Graduate  | Develop user-facing web interfaces.        | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB9    | Tesla                   | Negeri Sembilan | AI Engineer               | We are a client looking for an AI Engineer.               | Recent Graduate  | Develop and train machine learning models. | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB8    | Amazon                  | Johor           | Cloud Engineer            | We are a client looking for a Cloud Engineer.             | Recent Graduate  | Design and implement cloud solutions.      | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB7    | Microsoft               | Kuala Lumpur    | Data Analyst              | We are a client looking for a Data Analyst.               | Recent Graduate  | Collect, analyze, and visualize data.      | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB6    | Google                  | Kelantan        | Backend Developer         | We are a client looking for a Backend Developer.          | Recent Graduate  | Develop server-side logic and APIs.        | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB5    | Tesla                   | Negeri Sembilan | Junior Software Engineer  | We are a client looking for a Junior Software Engineer.   | Recent Graduate  | Assist with software development.          | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB4    | Amazon                  | Johor           | Junior Software Engineer  | We are a client looking for a Junior Software Engineer.   | Recent Graduate  | Assist with software development.          | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB3    | Microsoft               | Kuala Lumpur    | Junior Software Engineer  | We are a client looking for a Junior Software Engineer.   | Recent Graduate  | Assist with software development.          | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB2    | Google                  | Kelantan        | Junior Software Engineer  | We are a client looking for a Junior Software Engineer.   | Recent Graduate  | Assist with software development.          | 1,000.00 | 1-4        | 1.000 | Full Time     |
| JB1    | Google                  | Kelantan        | Software Engineer         | We are a client looking for a Software Engineer.          | Recent Graduate  | Develop and maintain web applications.     | 1,000.00 | 1-4        | 1.000 | Full Time     |

Diagram 6.6 : Diagram of Display All Job Posting After Sorting by Job ID with Descending

| Job ID | Company Name            | Location        | Position Title            |
|--------|-------------------------|-----------------|---------------------------|
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Quality Assurance         |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Data Analyst              |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Software Engineer         |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Overworked Intern         |
| JB25   | Blue Ocean              | Penang          | Software Developer        |
| JB24   | Microsoft               | Kuala Lumpur    | Software Developer        |
| JB23   | Otaku Coders            | Kuala Lumpur    | Software Developer        |
| JB22   | Amazon                  | Johor           | Software Developer        |
| JB21   | Amazon                  | Johor           | Cloud Developer           |
| JB20   | Otaku Coders            | Kuala Lumpur    | Software Tester           |
| JB19   | Otaku Coders            | Kuala Lumpur    | Database Engineer         |
| JB18   | Ani-Mining              | Kelantan        | Database Engineer         |
| JB17   | Intelligent Systems Co. | Johor           | Software Engineer         |
| JB16   | Amazon                  | Johor           | Data Engineer             |
| JB15   | Microsoft               | Kuala Lumpur    | Security Analyst          |
| JB14   | Amazon                  | Johor           | QA Tester                 |
| JB13   | Google                  | Kelantan        | DevOps Engineer           |
| JB12   | Tesla                   | Negeri Sembilan | Embedded Systems Engineer |
| JB11   | Google                  | Kelantan        | Site Reliability Engineer |
| JB10   | Facebook                | Kedah           | Frontend Developer        |
| JB9    | Tesla                   | Negeri Sembilan | AI Engineer               |
| JB8    | Amazon                  | Johor           | Cloud Engineer            |
| JB7    | Microsoft               | Kuala Lumpur    | Data Analyst              |
| JB6    | Google                  | Kelantan        | Backend Developer         |
| JB5    | Tesla                   | Negeri Sembilan | Junior Software Engineer  |
| JB4    | Amazon                  | Johor           | Junior Software Engineer  |
| JB3    | Microsoft               | Kuala Lumpur    | Junior Software Engineer  |
| JB2    | Google                  | Kelantan        | Junior Software Engineer  |
| JB1    | Google                  | Kelantan        | Software Engineer         |

Diagram 6.6.1 : Diagram of Display All Job Posting After Sorting by Job ID with Descending( Zoom In)

### Option 3 : Sort By Company Name Sort Ascending:

| Job ID | Company Name            | Location        | Position Title            | Job Description                                                                 | Job Requirements                 | Salary  | Hours | Days  | Position Type |
|--------|-------------------------|-----------------|---------------------------|---------------------------------------------------------------------------------|----------------------------------|---------|-------|-------|---------------|
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Quality Assurance         | Working for and only able to join and before Register to get and work on a task | Working and maintain existing DB | 1000.00 | 10    | 10.00 | 1000.00       |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Data Analyst              | Working for and only able to join and before Register to get and work on a task | Work with API and documentation  | 1000.00 | 10    | 10.00 | 1000.00       |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Software Engineer         | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Overworked Intern         | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB25   | Blue Ocean              | Penang          | Software Developer        | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB24   | Microsoft               | Kuala Lumpur    | Software Developer        | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB23   | Otaku Coders            | Kuala Lumpur    | Software Developer        | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB22   | Amazon                  | Johor           | Software Developer        | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB21   | Amazon                  | Johor           | Cloud Developer           | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB20   | Otaku Coders            | Kuala Lumpur    | Software Tester           | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB19   | Otaku Coders            | Kuala Lumpur    | Database Engineer         | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB18   | Ani-Mining              | Kelantan        | Database Engineer         | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB17   | Intelligent Systems Co. | Johor           | Software Engineer         | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB16   | Amazon                  | Johor           | Data Engineer             | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB15   | Microsoft               | Kuala Lumpur    | Security Analyst          | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB14   | Amazon                  | Johor           | QA Tester                 | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB13   | Google                  | Kelantan        | DevOps Engineer           | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB12   | Tesla                   | Negeri Sembilan | Embedded Systems Engineer | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB11   | Google                  | Kelantan        | Site Reliability Engineer | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB10   | Facebook                | Kedah           | Frontend Developer        | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB9    | Tesla                   | Negeri Sembilan | AI Engineer               | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB8    | Amazon                  | Johor           | Cloud Engineer            | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB7    | Microsoft               | Kuala Lumpur    | Data Analyst              | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB6    | Google                  | Kelantan        | Backend Developer         | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB5    | Tesla                   | Negeri Sembilan | Junior Software Engineer  | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB4    | Amazon                  | Johor           | Junior Software Engineer  | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB3    | Microsoft               | Kuala Lumpur    | Junior Software Engineer  | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB2    | Google                  | Kelantan        | Junior Software Engineer  | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |
| JB1    | Google                  | Kelantan        | Software Engineer         | Working for and only able to join and before Register to get and work on a task | Write and maintain code using    | 1000.00 | 10    | 10.00 | 1000.00       |

Diagram 6.7 : Diagram of Display All Job Posting After Sorting by Company Name with Ascending

| Job ID | Company Name            | Location        |
|--------|-------------------------|-----------------|
| JB4    | Amazon                  | Johor           |
| JB8    | Amazon                  | Johor           |
| JB14   | Amazon                  | Johor           |
| JB16   | Amazon                  | Johor           |
| JB21   | Amazon                  | Johor           |
| JB22   | Amazon                  | Johor           |
| JB18   | Ani-Mining              | Kelantan        |
| JB25   | Blue Ocean              | Penang          |
| JB26   | Exploitative Company Z  | Kuala Lumpur    |
| JB27   | Exploitative Company Z  | Kuala Lumpur    |
| JB28   | Exploitative Company Z  | Kuala Lumpur    |
| JB29   | Exploitative Company Z  | Kuala Lumpur    |
| JB10   | Facebook                | Kedah           |
| JB1    | Google                  | Kelantan        |
| JB2    | Google                  | Kelantan        |
| JB6    | Google                  | Kelantan        |
| JB11   | Google                  | Kelantan        |
| JB13   | Google                  | Kelantan        |
| JB17   | Intelligent Systems Co. | Johor           |
| JB3    | Microsoft               | Kuala Lumpur    |
| JB7    | Microsoft               | Kuala Lumpur    |
| JB15   | Microsoft               | Kuala Lumpur    |
| JB24   | Microsoft               | Kuala Lumpur    |
| JB19   | Otaku Coders            | Kuala Lumpur    |
| JB20   | Otaku Coders            | Kuala Lumpur    |
| JB23   | Otaku Coders            | Kuala Lumpur    |
| JB5    | Tesla                   | Negeri Sembilan |
| JB9    | Tesla                   | Negeri Sembilan |
| JB12   | Tesla                   | Negeri Sembilan |

Diagram 6.7 : Diagram of Display All Job Posting After Sorting by Company Name with Ascending (Zoom In)

Sort Descending:

| Job ID | Company Name            | Location        | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
|--------|-------------------------|-----------------|-----------------|------------------|----------------------|------------|----------|------------|----------|----------|
| JB12   | Tesla                   | Negeri Sembilan | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB9    | Tesla                   | Negeri Sembilan | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB5    | Tesla                   | Negeri Sembilan | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB23   | Otaku Coders            | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB20   | Otaku Coders            | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB19   | Otaku Coders            | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB24   | Microsoft               | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB15   | Microsoft               | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB7    | Microsoft               | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB3    | Microsoft               | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB17   | Intelligent Systems Co. | Johor           | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB13   | Google                  | Kelantan        | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB6    | Google                  | Kelantan        | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB2    | Google                  | Kelantan        | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB1    | Google                  | Kelantan        | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB10   | Facebook                | Kedah           | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB25   | Blue Ocean              | Penang          | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB18   | Ani-Mining              | Kelantan        | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB22   | Amazon                  | Johor           | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB21   | Amazon                  | Johor           | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB16   | Amazon                  | Johor           | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB14   | Amazon                  | Johor           | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB8    | Amazon                  | Johor           | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |
| JB4    | Amazon                  | Johor           | Job Description | Job Requirements | Job Responsibilities | Job Salary | Job Type | Job Status | Job Date | Job Type |

Diagram 6.8 : Diagram of Display All Job Posting After Sorting by Company Name with Descending

| Job ID | Company Name            | Location        | Position Title            |
|--------|-------------------------|-----------------|---------------------------|
| JB12   | Tesla                   | Negeri Sembilan | Embedded Systems Engineer |
| JB9    | Tesla                   | Negeri Sembilan | AI Engineer               |
| JB5    | Tesla                   | Negeri Sembilan | Junior Software Engineer  |
| JB23   | Otaku Coders            | Kuala Lumpur    | Software Developer        |
| JB20   | Otaku Coders            | Kuala Lumpur    | Software Tester           |
| JB19   | Otaku Coders            | Kuala Lumpur    | Database Engineer         |
| JB24   | Microsoft               | Kuala Lumpur    | Software Developer        |
| JB15   | Microsoft               | Kuala Lumpur    | Security Analyst          |
| JB7    | Microsoft               | Kuala Lumpur    | Data Analyst              |
| JB3    | Microsoft               | Kuala Lumpur    | Junior Software Engineer  |
| JB17   | Intelligent Systems Co. | Johor           | Software Engineer         |
| JB13   | Google                  | Kelantan        | DevOps Engineer           |
| JB11   | Google                  | Kelantan        | Site Reliability Engineer |
| JB6    | Google                  | Kelantan        | Backend Developer         |
| JB2    | Google                  | Kelantan        | Junior Software Engineer  |
| JB1    | Google                  | Kelantan        | Software Engineer         |
| JB10   | Facebook                | Kedah           | Frontend Developer        |
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Quality Assurance         |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Data Analyst              |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Software Engineer         |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Overworked Intern         |
| JB25   | Blue Ocean              | Penang          | Software Developer        |
| JB18   | Ani-Mining              | Kelantan        | Database Engineer         |
| JB22   | Amazon                  | Johor           | Software Developer        |
| JB21   | Amazon                  | Johor           | Cloud Developer           |
| JB16   | Amazon                  | Johor           | Data Engineer             |
| JB14   | Amazon                  | Johor           | QA Tester                 |
| JB8    | Amazon                  | Johor           | Cloud Engineer            |
| JB4    | Amazon                  | Johor           | Junior Software Engineer  |

*Diagram 6.8.1 : Diagram of Display All Job Posting After Sorting by Company Name with Descending (Zoom In )*

Option 4 : Sort By Location  
Sort Ascending:

*Diagram 6.9 : Diagram of Display All Job Posting After Sorting by Location with Ascending*

| Job ID | Company Name            | Location        | Position Title            |
|--------|-------------------------|-----------------|---------------------------|
| JB4    | Amazon                  | Johor           | Junior Software Engineer  |
| JB8    | Amazon                  | Johor           | Cloud Engineer            |
| JB14   | Amazon                  | Johor           | QA Tester                 |
| JB16   | Amazon                  | Johor           | Data Engineer             |
| JB17   | Intelligent Systems Co. | Johor           | Software Engineer         |
| JB21   | Amazon                  | Johor           | Cloud Developer           |
| JB22   | Amazon                  | Johor           | Software Developer        |
| JB10   | Facebook                | Kedah           | Frontend Developer        |
| JB1    | Google                  | Kelantan        | Software Engineer         |
| JB2    | Google                  | Kelantan        | Junior Software Engineer  |
| JB6    | Google                  | Kelantan        | Backend Developer         |
| JB11   | Google                  | Kelantan        | Site Reliability Engineer |
| JB13   | Google                  | Kelantan        | DevOps Engineer           |
| JB18   | Ani-Mining              | Kelantan        | Database Engineer         |
| JB3    | Microsoft               | Kuala Lumpur    | Junior Software Engineer  |
| JB7    | Microsoft               | Kuala Lumpur    | Data Analyst              |
| JB15   | Microsoft               | Kuala Lumpur    | Security Analyst          |
| JB19   | Otaku Coders            | Kuala Lumpur    | Database Engineer         |
| JB20   | Otaku Coders            | Kuala Lumpur    | Software Tester           |
| JB23   | Otaku Coders            | Kuala Lumpur    | Software Developer        |
| JB24   | Microsoft               | Kuala Lumpur    | Software Developer        |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Overworked Intern         |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Software Engineer         |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Data Analyst              |
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Quality Assurance         |
| JB5    | Tesla                   | Negeri Sembilan | Junior Software Engineer  |
| JB9    | Tesla                   | Negeri Sembilan | AI Engineer               |
| JB12   | Tesla                   | Negeri Sembilan | Embedded Systems Engineer |
| JB25   | Blue Ocean              | Penang          | Software Developer        |

*Diagram 6.9.1 : Diagram of Display All Job Posting After Sorting by Location with Ascending*

### Sort Descending:

[illegible]

*Diagram 6.10 : Diagram of Display All Job Posting After Sorting by Location with Descending*

| Job ID | Company Name            | Location        | Position Title            |
|--------|-------------------------|-----------------|---------------------------|
| JB25   | Blue Ocean              | Penang          | Software Developer        |
| JB12   | Tesla                   | Negeri Sembilan | Embedded Systems Engineer |
| JB9    | Tesla                   | Negeri Sembilan | AI Engineer               |
| JB5    | Tesla                   | Negeri Sembilan | Junior Software Engineer  |
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Quality Assurance         |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Data Analyst              |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Software Engineer         |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Overworked Intern         |
| JB24   | Microsoft               | Kuala Lumpur    | Software Developer        |
| JB23   | Otaku Coders            | Kuala Lumpur    | Software Developer        |
| JB20   | Otaku Coders            | Kuala Lumpur    | Software Tester           |
| JB19   | Otaku Coders            | Kuala Lumpur    | Database Engineer         |
| JB15   | Microsoft               | Kuala Lumpur    | Security Analyst          |
| JB7    | Microsoft               | Kuala Lumpur    | Data Analyst              |
| JB3    | Microsoft               | Kuala Lumpur    | Junior Software Engineer  |
| JB18   | Ani-Mining              | Kelantan        | Database Engineer         |
| JB13   | Google                  | Kelantan        | DevOps Engineer           |
| JB11   | Google                  | Kelantan        | Site Reliability Engineer |
| JB6    | Google                  | Kelantan        | Backend Developer         |
| JB2    | Google                  | Kelantan        | Junior Software Engineer  |
| JB1    | Google                  | Kelantan        | Software Engineer         |
| JB10   | Facebook                | Kedah           | Frontend Developer        |
| JB22   | Amazon                  | Johor           | Software Developer        |
| JB21   | Amazon                  | Johor           | Cloud Developer           |
| JB17   | Intelligent Systems Co. | Johor           | Software Engineer         |
| JB16   | Amazon                  | Johor           | Data Engineer             |
| JB14   | Amazon                  | Johor           | QA Tester                 |
| JB8    | Amazon                  | Johor           | Cloud Engineer            |
| JB4    | Amazon                  | Johor           | Junior Software Engineer  |

Diagram 6.10.1 : Diagram of Display All Job Posting After Sorting by Location with Descending

The Sort Job Posting Function in the Job Management Module allows users to sort job posts in an ascending and descending order based on criteria such as the company name, company location, job id and the salary. It is useful when users want to quickly access and compare between job posts.

## 7.0 Delete All Function :

| Job ID | Company Name            | Location        | Position Title            | Job Description                                                                   | Job Requirements                                            | Salary   | Position | DOB   | Position Page |
|--------|-------------------------|-----------------|---------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------|----------|----------|-------|---------------|
| JB25   | Blue Ocean              | Penang          | Software Developer        | Develop and maintain software applications for various clients.                   | Minimum 2 years experience in software development.         | 10000.00 | 1.0      | 1.000 | 1.000         |
| JB12   | Tesla                   | Negeri Sembilan | Embedded Systems Engineer | Design and develop embedded systems for automotive applications.                  | Minimum 3 years experience in embedded systems development. | 12000.00 | 1.2      | 1.000 | 1.000         |
| JB9    | Tesla                   | Negeri Sembilan | AI Engineer               | Research and develop artificial intelligence algorithms for various applications. | Minimum 4 years experience in AI research and development.  | 15000.00 | 1.5      | 1.000 | 1.000         |
| JB5    | Tesla                   | Negeri Sembilan | Junior Software Engineer  | Assist in the development and testing of software applications.                   | Minimum 1 year experience in software development.          | 8000.00  | 1.0      | 1.000 | 1.000         |
| JB29   | Exploitative Company Z  | Kuala Lumpur    | Quality Assurance         | Perform quality assurance testing on software applications.                       | Minimum 2 years experience in quality assurance testing.    | 7000.00  | 1.0      | 1.000 | 1.000         |
| JB28   | Exploitative Company Z  | Kuala Lumpur    | Data Analyst              | Analyze and interpret data to provide insights and recommendations.               | Minimum 2 years experience in data analysis.                | 9000.00  | 1.0      | 1.000 | 1.000         |
| JB27   | Exploitative Company Z  | Kuala Lumpur    | Software Engineer         | Develop and maintain software applications for various clients.                   | Minimum 2 years experience in software development.         | 10000.00 | 1.0      | 1.000 | 1.000         |
| JB26   | Exploitative Company Z  | Kuala Lumpur    | Overworked Intern         | Assist in the development and testing of software applications.                   | Minimum 1 year experience in software development.          | 6000.00  | 1.0      | 1.000 | 1.000         |
| JB24   | Microsoft               | Kuala Lumpur    | Software Developer        | Develop and maintain software applications for various clients.                   | Minimum 2 years experience in software development.         | 11000.00 | 1.0      | 1.000 | 1.000         |
| JB23   | Otaku Coders            | Kuala Lumpur    | Software Developer        | Develop and maintain software applications for various clients.                   | Minimum 2 years experience in software development.         | 9500.00  | 1.0      | 1.000 | 1.000         |
| JB20   | Otaku Coders            | Kuala Lumpur    | Software Tester           | Perform quality assurance testing on software applications.                       | Minimum 2 years experience in quality assurance testing.    | 7500.00  | 1.0      | 1.000 | 1.000         |
| JB19   | Otaku Coders            | Kuala Lumpur    | Database Engineer         | Design and develop database systems for various applications.                     | Minimum 3 years experience in database development.         | 10500.00 | 1.0      | 1.000 | 1.000         |
| JB15   | Microsoft               | Kuala Lumpur    | Security Analyst          | Monitor and analyze security threats to protect organizational assets.            | Minimum 3 years experience in security analysis.            | 13000.00 | 1.0      | 1.000 | 1.000         |
| JB7    | Microsoft               | Kuala Lumpur    | Data Analyst              | Analyze and interpret data to provide insights and recommendations.               | Minimum 2 years experience in data analysis.                | 9500.00  | 1.0      | 1.000 | 1.000         |
| JB3    | Microsoft               | Kuala Lumpur    | Junior Software Engineer  | Assist in the development and testing of software applications.                   | Minimum 1 year experience in software development.          | 8500.00  | 1.0      | 1.000 | 1.000         |
| JB18   | Ani-Mining              | Kelantan        | Database Engineer         | Design and develop database systems for various applications.                     | Minimum 3 years experience in database development.         | 10500.00 | 1.0      | 1.000 | 1.000         |
| JB13   | Google                  | Kelantan        | DevOps Engineer           | Manage and maintain the infrastructure for various applications.                  | Minimum 3 years experience in DevOps engineering.           | 14000.00 | 1.0      | 1.000 | 1.000         |
| JB11   | Google                  | Kelantan        | Site Reliability Engineer | Monitor and maintain the reliability of various applications.                     | Minimum 3 years experience in site reliability engineering. | 14500.00 | 1.0      | 1.000 | 1.000         |
| JB6    | Google                  | Kelantan        | Backend Developer         | Develop and maintain backend services for various applications.                   | Minimum 3 years experience in backend development.          | 13500.00 | 1.0      | 1.000 | 1.000         |
| JB2    | Google                  | Kelantan        | Junior Software Engineer  | Assist in the development and testing of software applications.                   | Minimum 1 year experience in software development.          | 8500.00  | 1.0      | 1.000 | 1.000         |
| JB1    | Google                  | Kelantan        | Software Engineer         | Develop and maintain software applications for various clients.                   | Minimum 2 years experience in software development.         | 10500.00 | 1.0      | 1.000 | 1.000         |
| JB10   | Facebook                | Kedah           | Frontend Developer        | Develop and maintain frontend services for various applications.                  | Minimum 3 years experience in frontend development.         | 12500.00 | 1.0      | 1.000 | 1.000         |
| JB22   | Amazon                  | Johor           | Software Developer        | Develop and maintain software applications for various clients.                   | Minimum 2 years experience in software development.         | 11500.00 | 1.0      | 1.000 | 1.000         |
| JB21   | Amazon                  | Johor           | Cloud Developer           | Develop and maintain cloud services for various applications.                     | Minimum 3 years experience in cloud development.            | 13500.00 | 1.0      | 1.000 | 1.000         |
| JB17   | Intelligent Systems Co. | Johor           | Software Engineer         | Develop and maintain software applications for various clients.                   | Minimum 2 years experience in software development.         | 10500.00 | 1.0      | 1.000 | 1.000         |
| JB16   | Amazon                  | Johor           | Data Engineer             | Design and develop data pipelines for various applications.                       | Minimum 3 years experience in data engineering.             | 13500.00 | 1.0      | 1.000 | 1.000         |
| JB14   | Amazon                  | Johor           | QA Tester                 | Perform quality assurance testing on software applications.                       | Minimum 2 years experience in quality assurance testing.    | 7500.00  | 1.0      | 1.000 | 1.000         |
| JB8    | Amazon                  | Johor           | Cloud Engineer            | Design and develop cloud services for various applications.                       | Minimum 3 years experience in cloud engineering.            | 13500.00 | 1.0      | 1.000 | 1.000         |
| JB4    | Amazon                  | Johor           | Junior Software Engineer  | Assist in the development and testing of software applications.                   | Minimum 1 year experience in software development.          | 8500.00  | 1.0      | 1.000 | 1.000         |

Diagram 7.0 : Diagram of Display All Job Posting Before Delete All Job Posting

Are you sure you want to delete all job postings? Yes = Y || No = N :

Y

Do you want to continue? (Y = Yes, N = No): N

| Job ID             | Company Name | Location | Position Title | Job Description |
|--------------------|--------------|----------|----------------|-----------------|
| The list is empty. |              |          |                |                 |

Diagram 7.1 : Diagram of Delete All Job Posting and Display Job Post List

The Delete All Job Posting Function in the Job Management Module allows deletion of all job posting in the system.

## Report Function:

## Job Posting Report

### 1. Report of Number of Job Posting for Each Company

#### 0. Exit

Option :

Diagram 8.0 : Diagram of Report Function Menu

Yusuf Abdul Rahnan University of Management and Technology

Internship System Program - Report of Job Posting for Each Company

Generated by 2025-04-18 15:19:23

| Job ID | Company Name            | Location        | Position Title            | Job Responsibilities                   | Requirements                                    | Salary   | Position Type |
|--------|-------------------------|-----------------|---------------------------|----------------------------------------|-------------------------------------------------|----------|---------------|
| J-001  | Google                  | Kelantan        | Software Engineer         | Develop and maintain existing SW       | Node.js, Java, SpringBoot, ASP.net, Flutter ... | 3000.00  | FULL TIME     |
| J-002  | Google                  | Kelantan        | Junior Software Engineer  | Develop and maintain existing SW       | Node.js, Java, SpringBoot, ASP.net, Flutter ... | 2000.00  | CONTRACT/TEMP |
| J-003  | Microsoft               | Kuala Lumpur    | Junior Software Engineer  | Develop and maintain existing SW       | ASP.net, .NET, Java, DevOps                     | 4000.00  | FULL TIME     |
| J-004  | Amazon                  | Johor           | Junior Software Engineer  | Develop and maintain existing SW       | ASP.net, .NET, Java, DevOps                     | 3400.00  | FULL TIME     |
| J-005  | Tealix                  | Negeri Sembilan | Junior Software Engineer  | Develop and maintain existing SW       | ASP.net, .NET, Java, DevOps                     | 6000.00  | FULL TIME     |
| J-006  | Google                  | Kelantan        | Backend Developer         | Build API and manage databases         | 3+ years in Java, Spring, MongoDB               | 4500.00  | PART TIME     |
| J-007  | Microsoft               | Kuala Lumpur    | Data Analyst              | Analyze and report key trends          | Excel, SQL, Python, R, ML, CSS                  | 3700.00  | FULL TIME     |
| J-008  | Amazon                  | Johor           | Cloud Engineer            | Work with AWS and microservices        | AWS, Docker, Kubernetes, Java                   | 5200.00  | FULL TIME     |
| J-009  | Tealix                  | Negeri Sembilan | AI Engineer               | Implement ML models in production      | Python, TensorFlow, PyTorch                     | 7000.00  | FULL TIME     |
| J-010  | Facebook                | Kedah           | Frontend Developer        | React, HTML, CSS, JavaScript           | Minimum 2 years experience in frontend dev      | 4000.00  | CONTRACT/TEMP |
| J-011  | Google                  | Kelantan        | Site Reliability Engineer | Monitoring, CI/CD, cloud ops           | Linux, Docker, Prometheus                       | 5800.00  | FULL TIME     |
| J-012  | Tealix                  | Negeri Sembilan | Embedded Systems Engineer | Write embedded C for avionics          | C, RTOS, hardware integration                   | 8000.00  | FULL TIME     |
| J-013  | Google                  | Kelantan        | DevOps Engineer           | Build robust deployment pipelines      | Docker, Kubernetes, Git, CI                     | 5100.00  | PART TIME     |
| J-014  | Amazon                  | Johor           | QA Tester                 | Write and automate test cases          | Testcase, JUnit, API testing                    | 3200.00  | CONTRACT/TEMP |
| J-015  | Microsoft               | Kuala Lumpur    | Security Analyst          | Security monitoring and response       | SIEM tools, network security                    | 6100.00  | FULL TIME     |
| J-016  | Amazon                  | Johor           | Data Engineer             | ETL and develop databases              | Python, SQL, C, C#, Rust, COBOL, ADP, MS        | 10000.00 | FULL TIME     |
| J-017  | Intelligent Systems Co. | Johor           | Software Engineer         | ETL and develop software applications  | Python, Java, C#, JavaScript                    | 10000.00 | FULL TIME     |
| J-018  | Anti-Mining             | Kelantan        | Database Engineer         | Develop and fix databases              | Python, SQL, AWS                                | 10000.00 | FULL TIME     |
| J-019  | Drake Coders            | Kuala Lumpur    | Database Engineer         | Develop and fix databases              | Python, SQL, AWS                                | 10000.00 | FULL TIME     |
| J-020  | Drake Coders            | Kuala Lumpur    | Software Tester           | Create unit test for applications      | Java, JavaScript, API Testing, Jira             | 800.00   | FULL TIME     |
| J-021  | Amazon                  | Johor           | Cloud Developer           | Develop and cloud networks             | SQL, AWS, Postgres, MongoDB, Oracle             | 1200.00  | FULL TIME     |
| J-022  | Amazon                  | Johor           | Software Developer        | Develop and maintain existing Software | Java, JavaScript                                | 400.00   | FULL TIME     |
| J-023  | Drake Coders            | Kuala Lumpur    | Software Developer        | Develop and maintain existing Software | Java, JavaScript, JS                            | 2000.00  | FULL TIME     |
| J-024  | Microsoft               | Kuala Lumpur    | Software Developer        | Develop and maintain existing Software | Java, JavaScript, JS, JS, Communication 101     | 1400.00  | FULL TIME     |
| J-025  | Blow Ocean              | Penang          | Software Developer        | Develop and maintain existing Software | Java, JavaScript                                | 9000.00  | FULL TIME     |
| J-026  | Exploitative Company X  | Kuala Lumpur    | Overworked Intern         | Develop and maintain existing Software | Rust                                            | 50.00    | FULL TIME     |
| J-027  | Exploitative Company X  | Kuala Lumpur    | Software Engineer         | Develop and maintain existing Software | Rust, JS                                        | 20.00    | FULL TIME     |
| J-028  | Exploitative Company X  | Kuala Lumpur    | Data Analyst              | Analyze data                           | Rust, JS, C                                     | 0.00     | FULL TIME     |
| J-029  | Exploitative Company X  | Kuala Lumpur    | Quality Assurance         | Assure Quality                         | Rust, JS, C, C#, Angular, Angular               | 20.00    | FULL TIME     |

Company : Job Posting Count

Google : 5

Microsoft : 4

Amazon : 6

Tealix : 3

Facebook : 1

Intelligent Systems Co. : 1

Anti-Mining : 1

Drake Coders : 3

Blow Ocean : 1

Exploitative Company X : 4

Company with largest Job Posting is Amazon : 6

Company with smallest Job Posting is Facebook : 1

Intelligent Systems Co. : 1

Anti-Mining : 1

Blow Ocean : 1

Diagram 8.1 : Diagram of Report of Number of Job Posting by Each Company

The Reporting Function in the Job Management Module allows users to generate and view the report related to job post and company. The report includes the detailed job posting data stored in the system, followed by the count of job posts of each of the companies. A graph is also generated to visualise representation of numbers of job postings by companies. Besides that, this function will also identify and display the most job post and the least job post company.

| Name        | Student ID | Prog / Tut.Grp | Signature |
|-------------|------------|----------------|-----------|
| Hea Hui Min | 24WMR08865 | RDSG4          | Min       |

## Module name: Arrange an Interview with Schedule Module

### Business Rules for Interview Management

1. Unique Interview ID
  - Every interview must have a unique ID in the format I0001, I0002...
  - The ID is auto-generated
2. Interview Scheduling
  - An interview must have a defined date and time, which must be set during the creation and follow the format(YYYY-MM-DD HH:MM)
3. Marking System
  - Pre-marking is auto-calculated as the number of match skills \* 10 and is determined based on the application at the time scheduling
  - Interview mark will be updated by the company after interview
  - Total marks will be calculated as : pre-marking + interview mark
4. Pass / Fail Criteria
  - A student is marked as pass or fail
  - Default pass status is false until explicitly updated
5. Comparison of interview
  - Interviews are comparable by total mark of the first candidate in the list
  - Sorting will be in the descending order(highest mark to lowest mark)

### 1. Source codes for Control classes

Filename : InterviewController.java

```
package Control;

import Adt.SortedDoublyLinkedList;
import Adt.SortedListInterface;
import Boundary.InternshipApplicationProgramUI;
import Boundary.InterviewUI;
import Dao.InternshipDAO;
import Entities.Application;
import Entities.Company;
import Entities.Interview;
import Utility.MessageUI;
import Utility.UserInput;
import java.time.LocalDateTime;
```

```

import java.time.format.DateTimeParseException;
import java.time.format.DateTimeFormatter;
import java.util.regex.*;

/**
 *
 * @author Hea Hui Min
 */
public class InterviewController {

 //ui module
 private static final InterviewUI InterviewUI = new InterviewUI();
 private static final UserInput userInput = new UserInput();
 private static final MessageUI MessageUI = new MessageUI();
 private static final InternshipApplicationProgramUI InternshipApplicationProgramUI =
new InternshipApplicationProgramUI();

 //list declared
 private SortedListInterface<Interview> interviewList = new
SortedListDoublyLinkedList<>();

 //DAO hardcoded data
 private static final InternshipDAO interviewDAO = new InternshipDAO(); //hardcoded
database

 //supported controller
 private ApplicationController ApplicationControl = ApplicationController.getInstance();

 public InterviewController() {
 interviewList = interviewDAO.initializeInterviewDAO();
 }

 public void runInterview() {
 InternshipApplicationProgramUI.clearScreen();
 InterviewUI.interviewLogo();
 displayInterviewMenu();
 }

 public void displayInterviewMenu() {
 int choice = 0;
 do {
 choice = InterviewUI.getMenuChoice();
 switch (choice) {
 case 0:
 System.err.println(MessageUI.exit());
 break;
 case 1:

```

```

 addNewInterview();
 displayInterview();
 break;
 case 2:
 displayInterview();
 break;
 case 3:
 deleteInterview();
 break;
 case 4:
 ///
 mergeInterview();
 break;
 case 5:
 assignInterviewMark();
 break;
 case 6:
 arrangeInterviewTime();
 break;
 case 7:
 displayFilterPassFail();
 break;
 case 8:
 generateRankSummaryReportByCompany();
 break;
 default:
 System.err.println(MessageUI.wrongInput());
 }
} while (choice != 0);
}

public void addNewInterview() {
 ApplicationControl.displayApplication();
 String appID = "";
 String interviewerName = "";
 String timeBooking = "";
 int year = 0;
 int month = 0;
 int day = 0;
 int hour = 0;
 int minute = 0;

 Application seletedApp = new Application();
 boolean validInput = false;
 Matcher matcher;

 while (!validInput) {
 appID = userInput.inputStringCleaner(InterviewUI.inputApplicationID());
 }
}

```

```

for (Application application : ApplicationControl.getApplicationList()) {

 if (application.getId().equalsIgnoreCase(appID)) {
 seletedApp = application;
 validInput = true;
 } else if ("XXX".equalsIgnoreCase(appID)) {
 return;
 }
}

if (!validInput) {
 System.err.println(MessageUI.wrongInput());
}
}

for (Interview interview : interviewList) { //same job cannot be apply or the job is
cancelled before
 for (Application application : interview.getApplicationList()) {
 if (seletedApp.equals(application)) {
 System.err.println(MessageUI.warning() + "This application already assign
interview!!!!");
 return;
 }
 }
}

interviewerName =
userInput.inputStringCleaner(InterviewUI.inputInterviewerName());
if ("XXX".equalsIgnoreCase(interviewerName)) {
 return;
}

validInput = false;
while (!validInput) {
 timeBooking = userInput.inputStringCleaner(InterviewUI.inputBookingTime());
 if (dateTimeValidator(timeBooking)) {

 matcher = dateTimeExtract(timeBooking);
 if (matcher.matches()) {

 year = Integer.parseInt(matcher.group(1));

 month = Integer.parseInt(matcher.group(2));

 day = Integer.parseInt(matcher.group(3));

 hour = Integer.parseInt(matcher.group(4));

 minute = Integer.parseInt(matcher.group(5));

```

```

 }
 validInput = true;
 } else if ("XXX".equalsIgnoreCase(timeBooking)) {

 return;
 }

 if (!validInput) {

 System.err.println(MessageUI.wrongInput());
 }
}

Interview newInterview = new Interview(seletedApp, year, month, day, hour, minute,
interviewerName);
//newInterview.setId(generateId(id++));
interviewList.insertAtTail(newInterview);

}

public void deleteInterview(){
 String interviewId =
userInput.inputStringCleaner(InterviewUI.inputDeleteInterviewId());

 boolean found = false;

 for(Interview interview : interviewList){
 if(interview.getId().equalsIgnoreCase(interviewId)){
 interviewList.deleteNode(interview);
 System.out.println("Interview Id : " + interviewId + "delete successful!!!");
 found = true;
 break;
 }
 }
 if(!found){
 System.out.println("Interview Id not found");
 }
}

public String getAllInterview() {
 String outputStr = "";
 for (int i = 0; i < interviewList.getSize(); i++) {
 outputStr += interviewList.atIndex(i) + "\n";
 }
 return outputStr;
}

```

```

public void displayInterview() {
 InterviewUI.listAllInterview(getAllInterview());
}

public boolean dateTimeValidator(String input) {
 String regex = "^(\\d{4})-(\\d{2})-(\\d{2}) (\\d{2}):\\d{2})$"; // include capturing
groups

 Pattern pattern = Pattern.compile(regex);
 Matcher matcher = pattern.matcher(input);

 if (!matcher.matches()) {
 System.err.println("Invalid format. Use YYYY-MM-DD HH:MM");
 return false;
 }
 return true;
}

public Matcher dateTimeExtract(String input) {
 String regex = "^(\\d{4})-(\\d{2})-(\\d{2}) (\\d{2}):\\d{2})$"; // include capturing
groups

 Pattern pattern = Pattern.compile(regex);
 Matcher matcher = pattern.matcher(input);
 return matcher;
}

public void mergeInterview() {
 String interviewID;
 boolean validInput = false;
 Interview seletedInterview = new Interview();
 Interview anotherInterview = new Interview();
 SortedListInterface<Interview> otherSame;

 displayInterview();
 while (!validInput) {
 interviewID =
userInput.inputStringCleaner(InterviewUI.inputInterviewIDforMerge());
 for (Interview interview : interviewList) {

 if (interview.getId().equalsIgnoreCase(interviewID)) {
 seletedInterview = interview;
 validInput = true;
 } else if ("XXX".equalsIgnoreCase(interviewID)) {
 return;
 }
 }
 }
 if (!validInput) {

```

```

 System.err.println(MessageUI.wrongInput());
 }
}

otherSame = sameJob(seletedInterview);
if (otherSame.getSize() == 0) {
 System.err.println(MessageUI.warning() + "No interview can be merge!!!");
 return;
}

InterviewUI.listCertainInterview(showList(otherSame));

validInput = false;
while (!validInput) {
 interviewID =
userInput.inputStringCleaner(InterviewUI.inputInterviewIDforMerge());

 for (Interview interview : otherSame) {

 if (interview.getId().equalsIgnoreCase(interviewID)) {
 anotherInterview = interview;
 validInput = true;

 } else if ("XXX".equalsIgnoreCase(interviewID)) {

 return;
 }
 }
 if (!validInput) {

 System.err.println(MessageUI.wrongInput());
 }
}

interviewList.deleteNode(anotherInterview);

seletedInterview.mergeWith(anotherInterview);
System.out.println("Successfully merged!!!");

}

public SortedListInterface<Interview> sameJob(Interview interview) {

 SortedListInterface<Interview> others = interviewList.cloneList();
 others.deleteNode(interview);

 SortedListInterface<Interview> otherSame = new SortedDoublyLinkedList<>();

```

```

 for (Interview other : others) {
 for (Application application : other.getApplicationList()) {
 if
(application.getJobPosting().equals(interview.getApplicationList().atIndex(0).getJobPostin
g())) {
 otherSame.insertSorted(other);
 }
 }
 }
 return otherSame;
 }

 public SortedListInterface<Interview> getInterviewList() {
 return interviewList;
 }

 public String showList(SortedListInterface<Interview> list) {
 String outputStr = "";
 for (Interview interview : list) {
 outputStr += interview.toString() + "\n";
 }
 return outputStr;
 }

 public void assignInterviewMark(){
 String interviewId = userInput.inputStringCleaner(InterviewUI.inputInterviewId());

 Interview selectedInterview = null;
 boolean validInput = false;

 for(Interview interview : interviewList){
 if(interview.getId().equalsIgnoreCase(interviewId)){
 selectedInterview = interview;
 validInput = true;
 break;
 }
 }

 if(!validInput){
 System.out.println(MessageUI.wrongInput());
 return;
 }

 int interviewMark = InterviewUI.inputInterviewIdScore();

 //Calculate the total mark
 int preMarking = selectedInterview.getPreMarking().atIndex(0);
 int totalMark = preMarking + interviewMark;
 }

```

```

selectedInterview.getInterviewMark().updateData(interviewMark, 0);

selectedInterview.getTotalMark().updateData(interviewMark, 0);

boolean pass = false;
while(true){
 String passInput = userInput.inputStringCleaner(InterviewUI.inputPassFail());
 if("Pass".equalsIgnoreCase(passInput)){
 pass = true;
 break;
 }else if("Fail".equalsIgnoreCase(passInput)){
 pass = false;
 break;
 }else{
 System.out.println("Please enter Pass or Fail !");
 }
}

selectedInterview.getPass().updateData(pass, 0);

System.out.println("Interview Mark is Updates. Total Mark is " + totalMark + ":" +
(pass ? "Pass" : "Fail"));

}

public void arrangeInterviewTime(){
 String interviewId = userInput.inputStringCleaner(InterviewUI.inputInterviewId());

 Interview selectedInterview = null;
 for(Interview interview : interviewList){
 if(interview.getId().equalsIgnoreCase(interviewId)){
 selectedInterview = interview;
 break;
 }
 }
 if(selectedInterview == null){
 System.out.println(MessageUI.wrongInput());
 return;
 }

 String timeInput = InterviewUI.inputBookingTime();
 LocalDateTime newTime =
LocalDateTime.parse(timeInput,DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm"));

 if (newTime.isBefore(LocalDateTime.now())) {
 System.out.println("The time cannot be in the past. Please enter again.");
 }
}

```

```

 return;
 }
 selectedInterview.setDateTime(newTime);

 System.out.println("The Interview Time is updated to: " +
newTime.format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")));
 }

 public void displayFilterPassFail() {
 SortedListInterface<Interview> filteredInterviewList = new
SortedDoublyLinkedList<>();
 String searchOption;

 searchOption = InterviewUI.inputPassFailResult();

 if (searchOption.equalsIgnoreCase("fail")) {
 searchOption = "false";
 } else if (searchOption.equalsIgnoreCase("pass")) {
 searchOption = "true";
 } else {
 System.out.println("Invalid input. Please enter 'pass' or 'fail'.");
 return;
 }

 filteredInterviewList = filteredInterviewList.extractByAttributes(interviewList,
searchOption);

 InterviewUI.displayInterviewHeader();
 for (Interview interview : filteredInterviewList) {
 String line = interview.toString()
 .replace("true", "Pass")
 .replace("false", "Fail");
 System.out.println(line);
 }
 InterviewUI.displayInterviewFooter(filteredInterviewList.getSize());
 }

 public void generateRankSummaryReportByCompany() {
 InternshipApplicationProgramUI.clearScreen();
 System.out.println("Generating Report...");

 DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");
 String currentDate = LocalDateTime.now().format(formatter);
 }

```

```

String[] companyNames = new String[100];
int[] passedCounts = new int[100];
int[] failedCounts = new int[100];
int companyCount = 0;

int totalInterviews = 0;

// HEADER
System.out.printf("=" .repeat(170) + "\n");
System.out.printf("\t" .repeat(7));
System.out.println("TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY");

System.out.printf("\t" .repeat(9));
System.out.println("INTERVIEW DATABASE SUBSYSTEM\n");

System.out.printf("\t" .repeat(9));
System.out.println("Summary Pass and Fail Report by All Company" .toUpperCase());
System.out.printf("\t" .repeat(8) + "-" .repeat(45) + "\n");

System.out.println("Generated at: " + currentDate);
System.out.printf("*" .repeat(170) + "\n");
System.out.printf("_" .repeat(170) + "\n");
System.out.println("TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY HIGHLY CONFIDENTIAL DOCUMENT");
System.out.printf("_" .repeat(170) + "\n");

// TABLE HEADER
System.out.printf("%-5s | %-15s | %-25s | %-30s | %-25s | %-20s | %-20s \n",
 "Rank", "Interview ID", "Student Name", "Company Name", "Position
Title", "Total Score", "Pass Status");

System.out.println("-----");
System.out.println("-----");

// Iterate through interviews and applications
int rank = 1;
for (Interview interview : interviewList) {
 if (interview != null && interview.getApplicationList() != null) {
 for (Application application : interview.getApplicationList()) {
 if (application != null && application.getJobPosting() != null) {
 String companyName =
application.getJobPosting().getCompany().getCompanyName();
 boolean isPass = interview.getPass().atIndex(0);
 totalInterviews++;
 }
 }
 }
}

```



```

 System.out.println("\n>> Passed Interview Summary Chart:");
 genBarChart(companyNames, passedCounts, companyCount, "Passed");

 System.out.println("\n>> Failed Interview Summary Chart:");
 genBarChart(companyNames, failedCounts, companyCount, "Failed");
 }

 public void genBarChart(String[] companyNames, int[] counts, int totalCompanies,
String title) {
 int maxHeight = 0;
 // Find the maximum count value to determine the chart height
 for (int i = 0; i < totalCompanies; i++) {
 maxHeight = Math.max(maxHeight, counts[i]);
 }

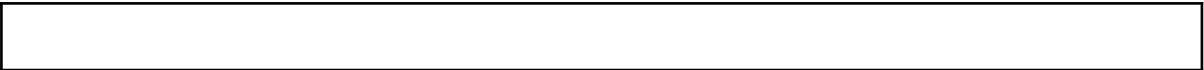
 System.out.println(" ^");
 for (int i = maxHeight; i > 0; i--) {
 System.out.printf("%2d | ", i);
 for (int j = 0; j < totalCompanies; j++) {
 if (counts[j] >= i) {
 System.out.printf("%-10s", "*");
 } else {
 System.out.printf("%-10s", " ");
 }
 }
 System.out.println();
 }

 System.out.print(" +");
 for (int i = 0; i < totalCompanies; i++) {
 System.out.print("-----");
 }
 System.out.println(">");

 System.out.print(" ");
 for (int i = 0; i < totalCompanies; i++) {
 String label = companyNames[i];
 if (label.length() > 8) label = label.substring(0, 8); // Truncate if the name is too
long
 System.out.printf("%-10s", label);
 }
 System.out.println();

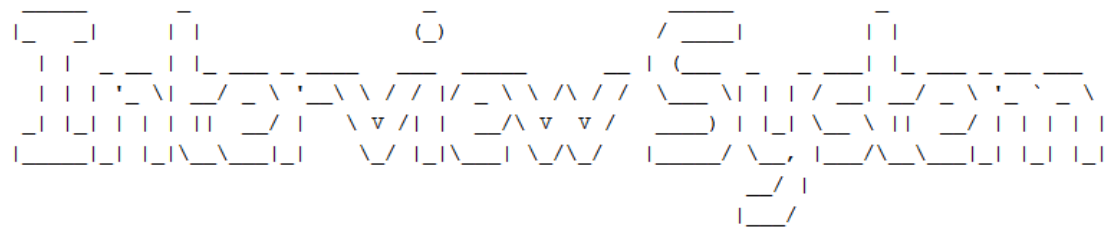
 System.out.println(" (" + title + ")");
 }
}

```



2. Screenshots

Interview Menu :



|               |                             |  |
|---------------|-----------------------------|--|
| +=====+       |                             |  |
|               | Arrange Interview Menu      |  |
| +=====+       |                             |  |
|               | Option No.   Description    |  |
| +-----+       |                             |  |
|               | 1   Create new Interview    |  |
|               | 2   Display all Interview   |  |
|               | 3   Delete Interview        |  |
|               | 4   Merge Interview         |  |
|               | 5   Assign Interview Mark   |  |
|               | 6   Manage Interview Time   |  |
|               | 7   Display Pass or Fail    |  |
|               | 8   Generate Summary Report |  |
|               | 0   Return to Main Menu     |  |
| +=====+       |                             |  |
| Enter choice: |                             |  |

Diagram of Display Arrange Interview Menu

Users are able to enter the choice (0-8) to select the function they need.

Create New Interview :

|                 |                             |  |
|-----------------|-----------------------------|--|
| +=====+         |                             |  |
|                 | Arrange Interview Menu      |  |
| +=====+         |                             |  |
|                 | Option No.   Description    |  |
| +-----+         |                             |  |
|                 | 1   Create new Interview    |  |
|                 | 2   Display all Interview   |  |
|                 | 3   Delete Interview        |  |
|                 | 4   Merge Interview         |  |
|                 | 5   Assign Interview Mark   |  |
|                 | 6   Manage Interview Time   |  |
|                 | 7   Display Pass or Fail    |  |
|                 | 8   Generate Summary Report |  |
|                 | 0   Return to Main Menu     |  |
| +=====+         |                             |  |
| Enter choice: 1 |                             |  |

| List of Application: |           |                           |          |           |                  |                     |         |                                                                                 |  |
|----------------------|-----------|---------------------------|----------|-----------|------------------|---------------------|---------|---------------------------------------------------------------------------------|--|
| App ID               | App Title | Company                   | Stud ID  | Stud Name | Matched Skills   | Submitted on        | Status  | Latest Log                                                                      |  |
| A0001                | J010      | Frontend Developer        | Facebook | 2401035   | Allison Jordan   | 14-04-2025 10:00:00 | PENDING | -STATUS- 10-04-2025 20:31:00] This application is submitted by Allison Jordan   |  |
| A0002                | J011      | Site Reliability Engineer | Google   | 2401021   | Alexander        | 15-04-2025 11:30:00 | PENDING | -STATUS- 10-04-2025 20:31:00] This application is submitted by Alexander        |  |
| A0003                | J012      | Embedded Systems Engineer | Twilio   | 2401025   | Aline Alvarado   | 15-04-2025 11:30:00 | PENDING | -STATUS- 10-04-2025 20:31:00] This application is submitted by Aline Alvarado   |  |
| A0004                | J013      | Devops Engineer           | Google   | 2401024   | Arie Shalton     | 17-04-2025 14:00:00 | PENDING | -STATUS- 10-04-2025 20:31:00] This application is submitted by Arie Shalton     |  |
| A0005                | J014      | QA Tester                 | Amazon   | 2401017   | Austrian Painter | 18-04-2025 13:15:00 | PENDING | -STATUS- 10-04-2025 20:31:00] This application is submitted by Austrian Painter |  |
| A0006                | J010      | Frontend Developer        | Facebook | 2401041   | Barton Underwood | 18-04-2025 20:31:00 | PENDING | -STATUS- 10-04-2025 20:31:00] This application is submitted by Barton Underwood |  |

Total Number of Application: 6

### Diagram of Application List

Press ENTER to continue

Which application u like to assign interview? Press XXX to quit.

Enter Application ID: A0001

Who will be the interviewer in this interview? Press XXX to quit.

Enter interviewer name: Michele

When this interview will be booked? Press XXX to quit.

Enter the date time in this format(YYYY-MM-DD HH:MM): 2025-05-25 15:00

### Diagram of User Enter

| List of Interview: |           |                           |          |           |                  |                     |           |            |  |
|--------------------|-----------|---------------------------|----------|-----------|------------------|---------------------|-----------|------------|--|
| ID                 | App Title | Company                   | Stud ID  | Stud Name | Matched Skills   | Submitted on        | Status    | Latest Log |  |
| I 10001            | J010      | Frontend Developer        | Facebook | 2401035   | Allison Jordan   | 14-04-2025 10:00:00 | 0 + 0-0   | false      |  |
| I 10002            | J012      | Embedded Systems Engineer | Twilio   | 2401025   | Aline Alvarado   | 15-04-2025 11:30:00 | 10 + 0-10 | false      |  |
| I 10003            | J013      | Devops Engineer           | Google   | 2401024   | Arie Shalton     | 16-04-2025 09:45:00 | 0 + 0-0   | false      |  |
| I 10004            | J014      | QA Tester                 | Amazon   | 2401017   | Austrian Painter | 17-04-2025 14:00:00 | 0 + 0-0   | false      |  |
| I 10005            | J010      | Frontend Developer        | Facebook | 2401041   | Barton Underwood | 18-04-2025 13:15:00 | 0 + 0-0   | false      |  |

### Diagram of List of Interview Before

| List of Interview: |           |                           |          |           |                  |                     |           |            |  |
|--------------------|-----------|---------------------------|----------|-----------|------------------|---------------------|-----------|------------|--|
| ID                 | App Title | Company                   | Stud ID  | Stud Name | Matched Skills   | Submitted on        | Status    | Latest Log |  |
| I 10001            | J010      | Frontend Developer        | Facebook | 2401035   | Allison Jordan   | 14-04-2025 10:00:00 | 0 + 0-0   | false      |  |
| I 10002            | J012      | Embedded Systems Engineer | Twilio   | 2401025   | Aline Alvarado   | 15-04-2025 11:30:00 | 10 + 0-10 | false      |  |
| I 10003            | J013      | Devops Engineer           | Google   | 2401024   | Arie Shalton     | 16-04-2025 09:45:00 | 0 + 0-0   | false      |  |
| I 10004            | J014      | QA Tester                 | Amazon   | 2401017   | Austrian Painter | 17-04-2025 14:00:00 | 0 + 0-0   | false      |  |
| I 10005            | J010      | Frontend Developer        | Facebook | 2401041   | Barton Underwood | 18-04-2025 13:15:00 | 0 + 0-0   | false      |  |
| I 10006            | J010      | Frontend Developer        | Facebook | 2401035   | Allison Jordan   | 15-04-2025 15:00:00 | 0 + 0-0   | false      |  |

### Diagram of List of Interview After

When the user selects 1, it will create a new interview from the Application list. Select the Application Id for which user want to create an interview for the person. Next, enter the interviewer's name, and finally, enter his interview time.

### Display All Interview :

| Accepage Interview Menu |                         |  |  |  |  |  |  |  |  |
|-------------------------|-------------------------|--|--|--|--|--|--|--|--|
| Option No.              | Description             |  |  |  |  |  |  |  |  |
| 1                       | Create new Interview    |  |  |  |  |  |  |  |  |
| 2                       | Display all Interview   |  |  |  |  |  |  |  |  |
| 3                       | Delete Interview        |  |  |  |  |  |  |  |  |
| 4                       | Merge Interview         |  |  |  |  |  |  |  |  |
| 5                       | Assign Interview Mark   |  |  |  |  |  |  |  |  |
| 6                       | Manage Interview Time   |  |  |  |  |  |  |  |  |
| 7                       | Display Pass or Fail    |  |  |  |  |  |  |  |  |
| 8                       | Generate Summary Report |  |  |  |  |  |  |  |  |
| 0                       | Return to Main Menu     |  |  |  |  |  |  |  |  |

Enter choice: 2

| List of Interview: |           |                           |          |           |                  |                     |           |            |  |
|--------------------|-----------|---------------------------|----------|-----------|------------------|---------------------|-----------|------------|--|
| ID                 | App Title | Company                   | Stud ID  | Stud Name | Matched Skills   | Submitted on        | Status    | Latest Log |  |
| I 10001            | J010      | Frontend Developer        | Facebook | 2401035   | Allison Jordan   | 14-04-2025 10:00:00 | 0 + 0-0   | false      |  |
| I 10002            | J012      | Embedded Systems Engineer | Twilio   | 2401025   | Aline Alvarado   | 15-04-2025 11:30:00 | 10 + 0-10 | false      |  |
| I 10003            | J013      | Devops Engineer           | Google   | 2401024   | Arie Shalton     | 16-04-2025 09:45:00 | 0 + 0-0   | false      |  |
| I 10004            | J014      | QA Tester                 | Amazon   | 2401017   | Austrian Painter | 17-04-2025 14:00:00 | 0 + 0-0   | false      |  |
| I 10005            | J010      | Frontend Developer        | Facebook | 2401041   | Barton Underwood | 18-04-2025 13:15:00 | 0 + 0-0   | false      |  |
| I 10006            | J010      | Frontend Developer        | Facebook | 2401035   | Allison Jordan   | 15-04-2025 15:00:00 | 0 + 0-0   | false      |  |

### Diagram of Display All Interview Record

Select 2 will display all the current interview records.

### Delete Interview :

```
|
| Arrange Interview Menu
|
|-----|
| Option No. | Description
|-----|
| 1 | Create new Interview
| 2 | Display all Interview
| 3 | Delete Interview
| 4 | Merge Interview
| 5 | Assign Interview Mark
| 6 | Manage Interview Time
| 7 | Display Pass or Fail
| 8 | Generate Summary Report
| 0 | Return to Main Menu
|
|-----|
Enter choice: 3

Enter the Interview Id than need to delete: I0006
Interview Id : I0006delete successful!!!

|
| Arrange Interview Menu
|
|-----|
| Option No. | Description
|-----|
| 1 | Create new Interview
| 2 | Display all Interview
| 3 | Delete Interview
| 4 | Merge Interview
| 5 | Assign Interview Mark
| 6 | Manage Interview Time
| 7 | Display Pass or Fail
| 8 | Generate Summary Report
| 0 | Return to Main Menu
|
|-----|
Enter choice: 2

List of Interview:
I0001	JB10	Frontend Developer	Facebook	2401035 Aileen Jordan	14-04-2025 10:00:00	0 + 0-0	false
I0002	JB12	Embedded Systems Engineer	Tesla	2401028 Aline Alvarado	15-04-2025 11:30:00	10 + 0-10	false
I0003	JB13	DevOps Engineer	Google	2401026 Amie Shelton	16-04-2025 09:45:00	0 + 0-0	false
I0004	JB14	QA Tester	Amazon	2401017 Austrian Painter	17-04-2025 14:00:00	0 + 0-0	false
I0005	JB10	Frontend Developer	Facebook	2401041 Barton Underwood	18-04-2025 13:15:00	0 + 0-0	false
```

Diagram of Delete Interview

Select 3 to access the delete function, which will delete the interview record. Enter the Interview ID that you want to delete, and if successful, it will display the delete successful message.

Merge Interview :

```
+=====+
Arrange Interview Menu
Option No.

1
2
3
4
5
6
7
8
0

Enter choice: 4

List of Interview:
I0001	JB10	Frontend Developer	Facebook	2401035 Aileen Jordan	14-04-2025 10:00:00	0 + 0-0	false
I0002	JB12	Embedded Systems Engineer	Tesla	2401028 Aline Alvarado	15-04-2025 11:30:00	10 + 0-10	false
I0003	JB13	DevOps Engineer	Google	2401026 Amie Shelton	16-04-2025 09:45:00	0 + 0-0	false
I0004	JB14	QA Tester	Amazon	2401017 Austrian Painter	17-04-2025 14:00:00	0 + 0-0	false
I0005	JB10	Frontend Developer	Facebook	2401041 Barton Underwood	18-04-2025 13:15:00	0 + 0-0	false
I0006	JB10	Frontend Developer	Facebook	2401035 Aileen Jordan	18-04-2025 13:15:00	0 + 0-0	false

Which interview u like to merge? Press XXX to quit.
Enter interview ID: I0001

List of Interview that able to be match:
| I0005 | JB10 | Frontend Developer | Facebook | 2401041 Barton Underwood | 18-04-2025 13:15:00 | 0 + 0-0 | false |

Which interview u like to merge? Press XXX to quit.
Enter interview ID: I0005
Successfully merged!!
```

Diagram of User to merge the Interview Record

```
List of Interview:
I0001	JB10	Frontend Developer	Facebook	2401035 Aileen Jordan,2401041 Barton Underwood	14-04-2025 10:00:00	0 + 0-0,0 + 0-0	false,false
I0002	JB12	Embedded Systems Engineer	Tesla	2401028 Aline Alvarado	15-04-2025 11:30:00	10 + 0-10	false
I0003	JB13	DevOps Engineer	Google	2401026 Amie Shelton	16-04-2025 09:45:00	0 + 0-0	false
I0004	JB14	QA Tester	Amazon	2401017 Austrian Painter	17-04-2025 14:00:00	0 + 0-0	false
I0006	JB10	Frontend Developer	Facebook	2401035 Aileen Jordan	18-04-2025 13:15:00	0 + 0-0	false
```

Diagram of updated

Select 4 will take you to the merge interview, where you can merge the same company into one row. For example, I0001 and I0005 interview with the same employer, allowing them to merge.

Assign Interview Mark :

```

Enter the Interview Id: I0002
Enter the Interview Marks: 80
Enter Pass or Fail : pass
Interview Mark is Updated, Total Mark is 90:Pass
=====
| Arrange Interview Menu |
|-----|-----|
Option No.	Description
1	Create new Interview
2	Display all Interview
3	Delete Interview
4	Merge Interview
5	Assign Interview Mark
6	Manage Interview Time
7	Display Pass or Fail
8	Generate Summary Report
9	Return to Main Menu
-----	-----
Enter choice: 2

```

```

List of Interview:
| I0001 | JB10 | Frontend Developer | Facebook | 2401035 Aileen Jordan | 14-04-2025 10:00:00 | 0 + 0-
| I0002 | JB12 | Embedded Systems Engineer | Tesla | 2401025 Alina Alvarado | 15-04-2025 11:30:00 | 10 + 8-
| I0003 | JB13 | DevOps Engineer | Google | 2401026 Amla Shelton | 16-04-2025 09:45:00 | 0 + 0-
| I0004 | JB14 | QA Tester | Amazon | 2401017 Austrian Painter | 17-04-2025 14:00:00 | 0 + 0-
| I0005 | JB10 | Frontend Developer | Facebook | 2401041 Barton Underwood | 18-04-2025 13:15:00 | 0 + 0-

```

### Diagram of User Assign the Interview Mark

Select 5 are able to assign the interview mark. Enter the Interview ID, followed by the interview mark and status you wish to provide. Then it will provide a notification about the overall mark and pass status.

### Arrange Time :

```

+=====+
| Arrange Interview Menu |
|-----+-----+
| Option No. | Description |
|-----+-----+
1	Create new Interview
2	Display all Interview
3	Delete Interview
4	Merge Interview
5	Assign Interview Mark
6	Manage Interview Time
7	Display Pass or Fail
8	Generate Summary Report
9	Return to Main Menu
-----+-----+	

```

Enter choice: 6

Enter the Interview Id: I0004

When this interview will be booked? Press XXX to quit.

Enter the date time in this format(YYYY-MM-DD HH:MM): 2025-12-12 15:00

The Interview Time is updated to: 2025-12-12 15:00

### Diagram of User to updated

```

| I0004 | JB14 | QA Tester | Amazon | 2401017 Austrian Painter | 17-04-2025 14:00:00 | 0 + 0- | False |

```

### Diagram Before Update

```

| I0004 | JB14 | QA Tester | Amazon | 2401017 Austrian Painter | 12-12-2025 15:00:00

```

### Diagram After Update

Select 6 allows users to change the interview time for a single person. First, input the Interview ID to find it, then enter the time that has to be changed in the format Year, Month, Day, Hour, Minutes.

### Filter Pass and Fail :

```

Enter the interview result to filter by (e.g., 'pass', 'fail')pass

| Interview ID | Job ID | Position | Company | Applicant | Date & Time | Marks (Pre + Interview + Total) | Result |

| I0002 | J001 | Embedded Systems Engineer | Tesla | PRADISDA Aline Alvarado | 16-04-2025 11:30:00 | 10 + 80=90 | Pass |

Total Interview: 1

```

### Diagram of Filter Pass Record

Select 7 allows the user to filter the records that they need to view. For example, when you input the pass, it will display the pass record; if you enter the fail, it will reveal all of the fail interview records.

Generate the summary report about the Pass and Fail Record by all Company :

Generating Report...

```

=====
TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY
INTERVIEW DATABASE SUBSYSTEM

SUMMARY PASS AND FAIL REPORT BY ALL COMPANY

Generated at: 2025-04-18 03:02:20
=====
TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY HIGHLY CONFIDENTIAL DOCUMENT
=====
Rank | Interview ID | Student Name | Company Name | Position Title | Total Score | Pass Status |

1 | I0003 | Amie Shelton | Google | DevOps Engineer | 80 | Pass |
2 | I0002 | Aline Alvarado | Tesla | Embedded Systems Engineer | 10 | Fail |
3 | I0001 | Aileen Jordan | Facebook | Frontend Developer | 0 | Fail |
4 | I0004 | Austrian Painter | Amazon | QA Tester | 0 | Fail |
5 | I0005 | Barton Underwood | Facebook | Frontend Developer | 0 | Fail |

>> Passed Interview Summary Chart:
^
1 | *
+----->
Google Tesla Facebook Amazon
(Passed)

>> Failed Interview Summary Chart:
^
2 |
1 | * * *
+----->
Google Tesla Facebook Amazon
(Failed)

```

### Diagram of Summary Report

Select 8 will provide a summary report of all the companies' pass and fail results, ranked by higher score. The bar chart below displays the summary charts for passed and failed interviews.